Understanding PSOC™ Edge memory usage training manual

About this document

Scope and purpose

This document is the training manual for “Understanding PSOC™ Edge memory usage” training. It provides detailed, step-by-step instructions for completing practical laboratory exercises that involve modifying code and customising data placement across different PSOC™ Edge memory types using ModusToolbox™ software.

Table of contents

About this document **1**
Table of contents **2**
1 Introduction **4**
2 Pre-requisites **4**
3 Required development tools **5**
4 Assigning code and data to custom linker sections **6**
4.1 Objective 6
4.2 Description 6
4.3 Project creation 6
4.4 Assigning code and data to linker sections 6
4.4.1 Part 1: Observe default placement of functions and variables 7
4.4.2 Part 2: Place code and data in different sections 12
4.4.3 Part 3: Placing code and data in a custom region 14
4.4.4 Other things to try 19
4.4.5 Key takeaways 19
4.5 Conclusion 19
5 Assigning a source file to a linker section **20**
5.1 Objective 20
5.2 Project creation 20
5.3 Description 20
5.4 Assigning files to linker sections 20
5.4.1 Update proj_cm55 to observe placement of code and data 20
5.4.2 Moving proj_cm55/main.c code 25
5.4.3 Other things to try 27
5.4.4 Key takeaways 28
5.5 Conclusion 28
6 Placing and executing code from RRAM **29**
6.1 Objective 29
6.2 Description 30
6.3 Project creation 30
6.4 Move all application images into RRAM 31
6.4.1 Changes in main.c file of the *cm33_ns* project 31
6.4.2 Memory personality update 31
6.4.3 Update CM55 App start address 33
6.4.4 Modify “boot_with_extended_boot.json” 34
6.4.5 Hardware (Boot switch) setting 36
6.4.6 Build load and run 36
6.4.7 Output 36
6.5 Moving the CM55 image to the external flash 37
6.5.1 Observe default memory placement in the CM55 project 37
6.5.2 Update the memory locations in the device configurator 39
6.5.3 Update CM55 App start address 39
6.5.4 Configure and enable the SMIF in the CM33_S user application 40
6.5.5 Modify “boot_with_extended_boot.json” 43
6.5.6 Build load and run 43
6.5.7 Output 44
6.6 Key takeaways 44

Table of contents

6.7 Conclusion 44
7 Placing and executing code from SRAM 45
7.1 Objective 45
7.2 Description 45
7.3 Project creation 45
7.4 Using the Edge Protect Bootloader to launch images from external flash 46
7.4.1 Copy the bootloader project into the application project 46
7.4.2 Modify the application 48
7.5 Using the Edge Protect Bootloader to load application images into SRAM 52
7.6 Output 54
7.7 Key takeaways 55
7.8 Conclusion 55
8 Low-power memory considerations 56
8.1 Objective 56
8.2 Description 56
8.3 Project creation 56
8.4 Baseline memory usage power consumption 57
8.5 Disabling SoCMEM 60
8.6 SoCMEM retention 61
8.7 SRAM retention 64
8.8 Output 68
8.9 Takeaways 69
8.10 Conclusion 69
9 Appendix A: Creating a PSOC™ Edge application in ModusToolbox™ 70
10 Appendix B: KIT_PSE84_EVAL details 73
Revision history 74
Disclaimer 75

1 Introduction

This manual provides instructions for creating, configuring, building, and running several code examples utilizing different features of PSOC™ Edge E84 memories.

2 Pre-requisites

Install the software and obtain the hardware listed in the Required development tools section.

This class will not cover basic concepts of ModusToolbox™ and PSOC™ Edge.

- For an introduction to PSOC™, including getting started with ModusToolbox™, go to: https://www.infineon.com/product-information/psocdeveloper
  - For PSOC™ Edge trainings, from beginner tutorials to advanced trainings, go to: PSOC™ Edge E84 Training Collection

3 Required development tools

- ModusToolbox™ software v3.7 or later
  - Recommended installation via ModusToolbox™ Setup tool
- Eclipse IDE for ModusToolbox™ v2025.8.0 or later
  - Recommended installation via ModusToolbox™ Setup tool
- Edge Protect Security Suite v1.6 or later
  - Installed by ModusToolbox™ Setup tool as a dependency to ModusToolbox™ v3.7
- ModusToolbox™ Programming Tools v1.7.0 or later
  - Installed by ModusToolbox™ Setup tool as a dependency to ModusToolbox™ v3.7
- Board support package (BSP)
  - KIT_PSE84_EVAL_EPC2 v1.0.0 or later (available with ModusToolbox™ v3.7)
- Serial terminal emulator
  - Eclipse IDE for ModusToolbox™ includes a serial terminal that includes Tera Term, PuTTY, and so on.
- PSOC™ Edge E84 Evaluation Kit (KIT_PSE84_EVAL)
  - Ensure device boots from external memory:
    - BOOT SW/SW6 in 'ON/HIGH' position
  - Disable alternate serial interface configuration:
    - Tristate/Not-Connected (NC) J20/J21 position
  - For more details, see Appendix B: KIT_PSE84_EVAL details.

4 Assigning code and data to custom linker sections

4.1 Objective

The objective of this exercise is to learn how to assign functions and variables to specific linker sections in a PSOC™ Edge MCU based project, and to place code from a specific source file into a dedicated linker section, thereby gaining control over the memory layout of your application and precisely allocating code and data to desired memory regions, using the PSOC™ Edge Hello World code example as a base application.

4.2 Description

When working with PSOC™ Edge MCU-based projects, you can take control of code and data placement in memory regions by leveraging the Memory Configurator tool, linker files, and macros provided.

In this lab, you will learn how to effectively allocate code and data to a dedicated linker section, improving memory allocation efficiency in your PSOC™ Edge MCU-based project.

This code example has a three-project structure, including: CM33 secure, CM33 non-secure, and CM55 projects. All three projects are programmed to the external QSPI flash and executed in XIP mode.

By default, the projects use the following memories:

Table 1 Default placement of Hello World code example

Project	Data	Code
CM33 non-secure	SRAM1	External flash
CM33 secure	SRAM0	External flash
CM55	System SRAM (SoCMEM)	External flash

4.3 Project creation

Follow steps 1-6 in Appendix A: Creating a PSOC™ Edge application in ModusToolbox™ to create a new application using the Hello World example. Use a descriptive project name, such as **pse_memory_ex1**.

4.4 Assigning code and data to linker sections

This exercise modifies the PSOC™ Edge Hello World code example to demonstrate linker section assignments.

The first section shows the default placement of variables and code, while the second part shows how to place them in different sections by using linker directives.

4.4.1 Part 1: Observe default placement of functions and variables

1. Open `proj_cm33_ns/main.c`
2. Declare new variables and a function as shown below

Code listing 1 Declaring variable and function on `proj_cm33_ns/main.c`

```
/*******************************************************************************
* Macros
*******************************************************************************/
#define BLINKY_LED_DELAY_MSEC       (1000U)

/* The timeout value in microseconds used to wait for CM55 core to be booted */
#define CM55_BOOT_WAIT_TIME_USEC    (10U)

/* App boot address for CM55 project */
#define CM55_APP_BOOT_ADDR          (CYMEM_CM33_0_m55_nvm_START + \
                                     CYBSP_MCUBOOT_HEADER_SIZE)

/*******************************************************************************
* Global variables
*******************************************************************************/
/* Volatile variable in RAM */
__attribute__((used))  volatile uint32_t testVar = 0;
__attribute__((used))  volatile uint32_t testVar2 = 0;

/*******************************************************************************
* Local functions
*******************************************************************************/
/* Function in non-volatile memory */
__attribute__((used)) void testFunction(void)
{
    testVar++;
    testVar2++;
}
```

Assigning code and data to custom linker sections

3. Add code to call `testFunction()` and show placement of variables and functions

Code listing 2 Printing address of functions and variables

```
printf("https://github.com/Infineon/"
       "Code-Examples-for-ModusToolbox-Software\r\n\n");

printf("address of variable (testVar): 0x%08X\n", (unsigned int) &testVar);
printf("address of variable (testVar2): 0x%08X\n", (unsigned int) &testVar2);
printf("address of function (main): 0x%08X\n", (unsigned int)  main);
printf("address of function (testFunction): 0x%08X\n", (unsigned int)
testFunction);

/* Enable CM55. */
/* CM55_APP_BOOT_ADDR must be updated if CM55 memory layout is changed.*/
Cy_SysEnableCM55(MXCM55, CM55_APP_BOOT_ADDR, CM55_BOOT_WAIT_TIME_USEC);

for(;;)
{
    testFunction();
    Cy_GPIO_Inv(CYBSP_USER_LED_PORT, CYBSP_USER_LED_PIN);
    Cy_SysLib_Delay(BLINKY_LED_DELAY_MSEC);
}
```

4. Build the application

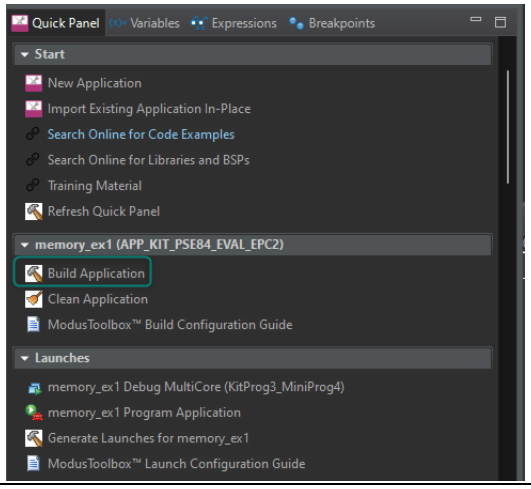


Figure 1 Build the application

5. ModusToolbox™ will create .map files for each of the three projects. Use the Plain Text Editor to observe the .map file for proj_cm33_ns

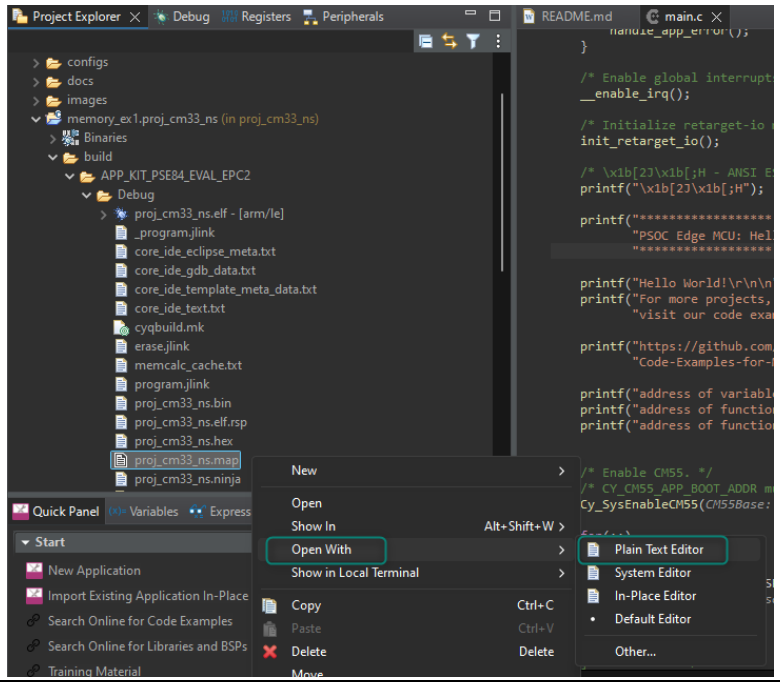


Figure 2 Observing generated .map files

6. The .map file will show the placement of all sections, variables and functions for the application. Observe the placement of *testVar, testVar2,* and *testFunction*

Code listing 3 Placement of variables and functions in the .map file

```
...
      .text.testFunction          0x08340f00          0x10
...
      .bss.testVar                0x240bd598          0x4
...
      .bss.testVar2               0x240bd59c          0x4
...
```

7. Program the application

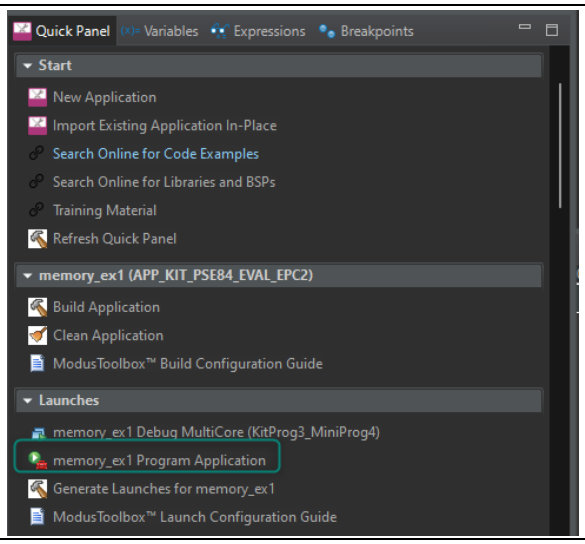


Figure 3 Programming application

8. Open a serial terminal at 115, 200 8-N-1 and observe the results

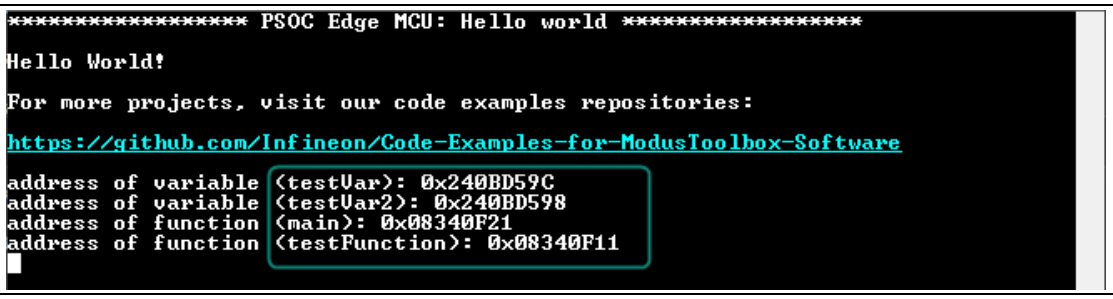


Figure 4 Terminal output showing default placement of variables and functions

9. ModusToolbox™ includes a **Device Configurator,** which can be used to configure the device and memory. We will not modify memory in this section, but the following steps show how to observe the default memory placement of the application

   a. Open the **Device Configurator** in the Quick Panel

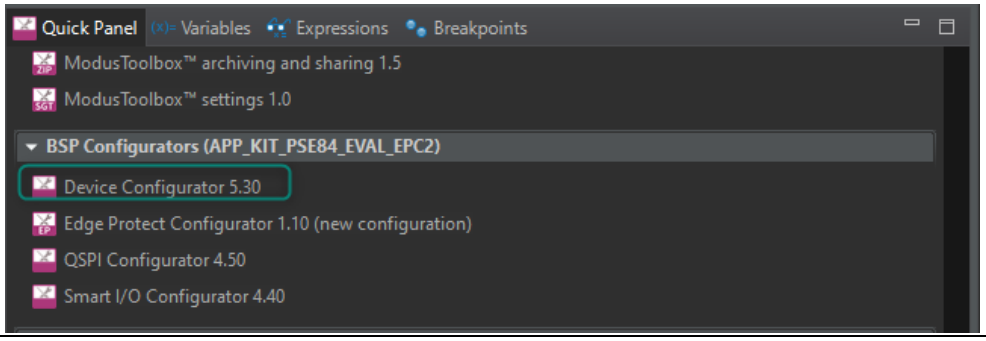


Figure 5 Opening device configurator

Assigning code and data to custom linker sections

b. Select the **Memory Tab**, then select **Memory Address View**, and make sure the CM33 core is selected. The lab modifies the non-secure CM33 application

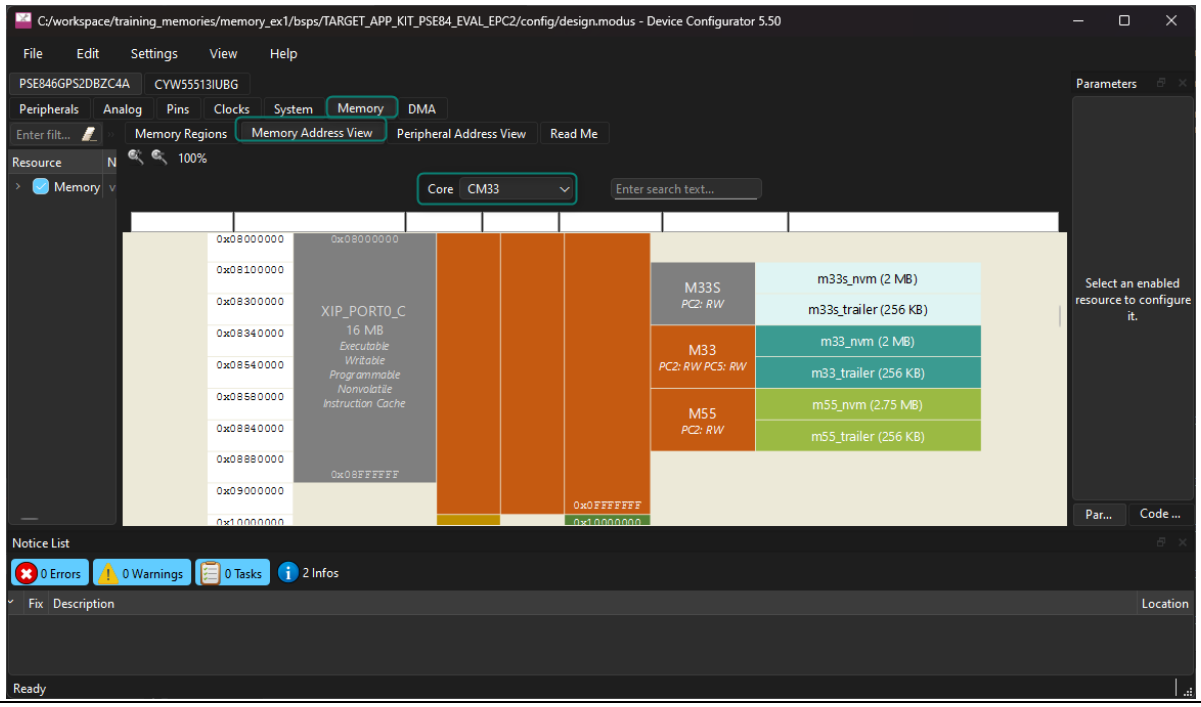


Figure 6 Memory address view

c. Observe the default placement of sections

Table 2 Memory locations in Memory Configurator

Variable/Function	Default memory location	Memory configurator Region ID
testVar	SRAM data region	m33_data
testVar2	SRAM data region	m33_data
testFunction	External flash region	m33_nvm

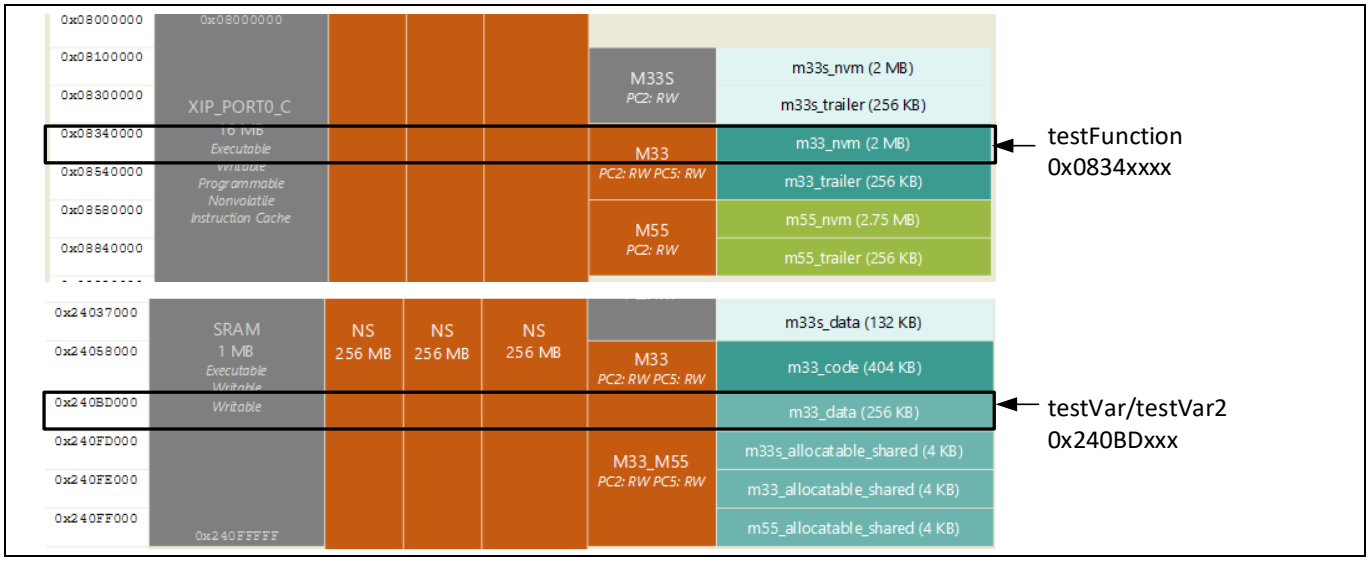


Figure 1 Placement of variables and functions in the memory configurator

4.4.2 Part 2: Place code and data in different sections

The first modification will move the "*testVar*" variable from its default location in m33_data SRAM to the m33_m55_system_shared SRAM memory region; "*testVar2*" to the m33_m55_shared region in System SRAM (SOCMEM); and "*testFunction*" from m33_nvm in external flash to the m33_code in SRAM.

1. The default memory placement can be observed in the linker files
   a. Open the m33_ns linker file at:
      <example>\bsps\TARGET_APP_KIT_PSE84_EVAL_EPC2\COMPONENT_CM33\TOOLCHAIN_GCC_ARM\pse84_ns_cm33.ld
   b. Note that multiple linker sections are placed in different regions. In the example Figure 2, the section .cy_ramfunc is executed from m33_code_sel, but placed in m33_nvm_sel. Initialization code will take care of copying it from non-volatile memory to SRAM

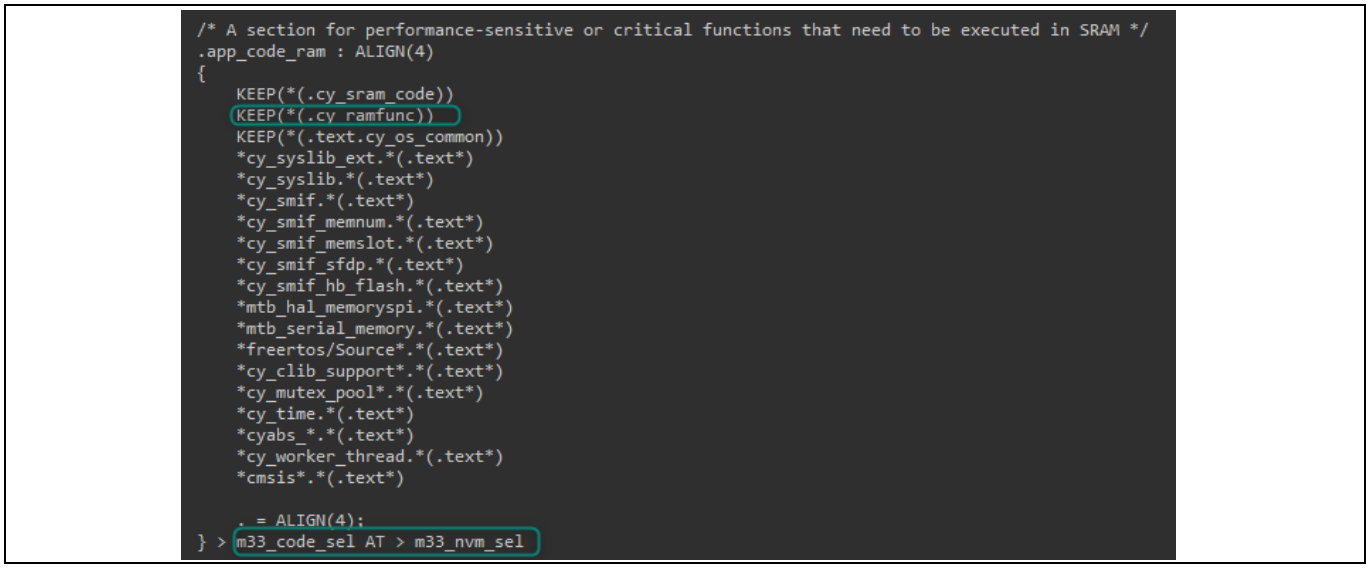


Figure 2 Placement of .cy_ramfunc

   c. We will use the following sections to place variables and functions

Table 3 Modified memory placement

Variable/Function	Memory Configurator Region ID	Memory region	Linker section
testVar	m33_allocatable_shared	SRAM	*.cy_sharedmem*
testVar2	m33_m55_shared	System SRAM	*.cy_shared_socmem*
testFunction	m33_code	SRAM	*.cy_ramfunc*

Assigning code and data to custom linker sections

2. Place the variables and functions in their new sections

Code listing 4 Placing variables and functions in memory sections

```
/*******************************************************************************
* Global variables
*******************************************************************************/
/* Volatile variable in RAM */
CY_SECTION(".cy_sharedmem") __attribute__((used))  volatile uint32_t testVar = 0;
CY_SECTION(".cy_shared_socmem") __attribute__((used))  volatile uint32_t testVar2 =
0;


/*******************************************************************************
* Local functions
*******************************************************************************/
/* Function in non-volatile memory */
CY_SECTION(".cy_ramfunc") __attribute__((used)) void testFunction(void)
{
    testVar++;
}
```

3. Build and reprogram the application
4. Observe the results in a terminal, and/or in the **.map** file

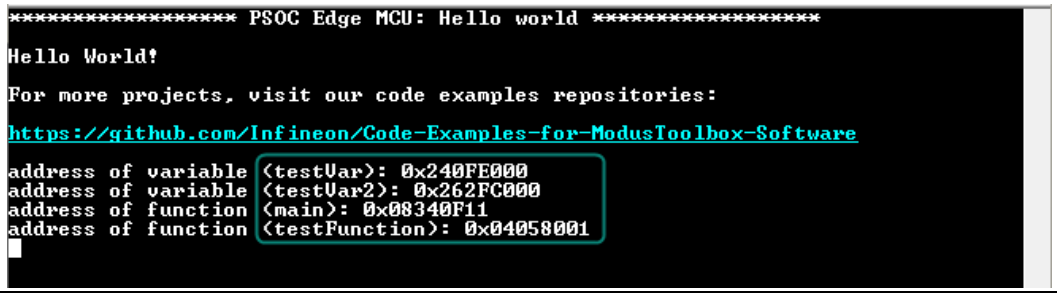


Figure 3 Terminal output with updated memory locations

5. Table 4 shows the modified memory locations

Table 4 Modified memory locations

Variable/Function	Default memory location	Updated memory location
testVar	SRAM data region	SRAM data region (shared)
testVar2	SRAM data region	System SRAM region (shared)
testFunction	External flash region	SRAM code region

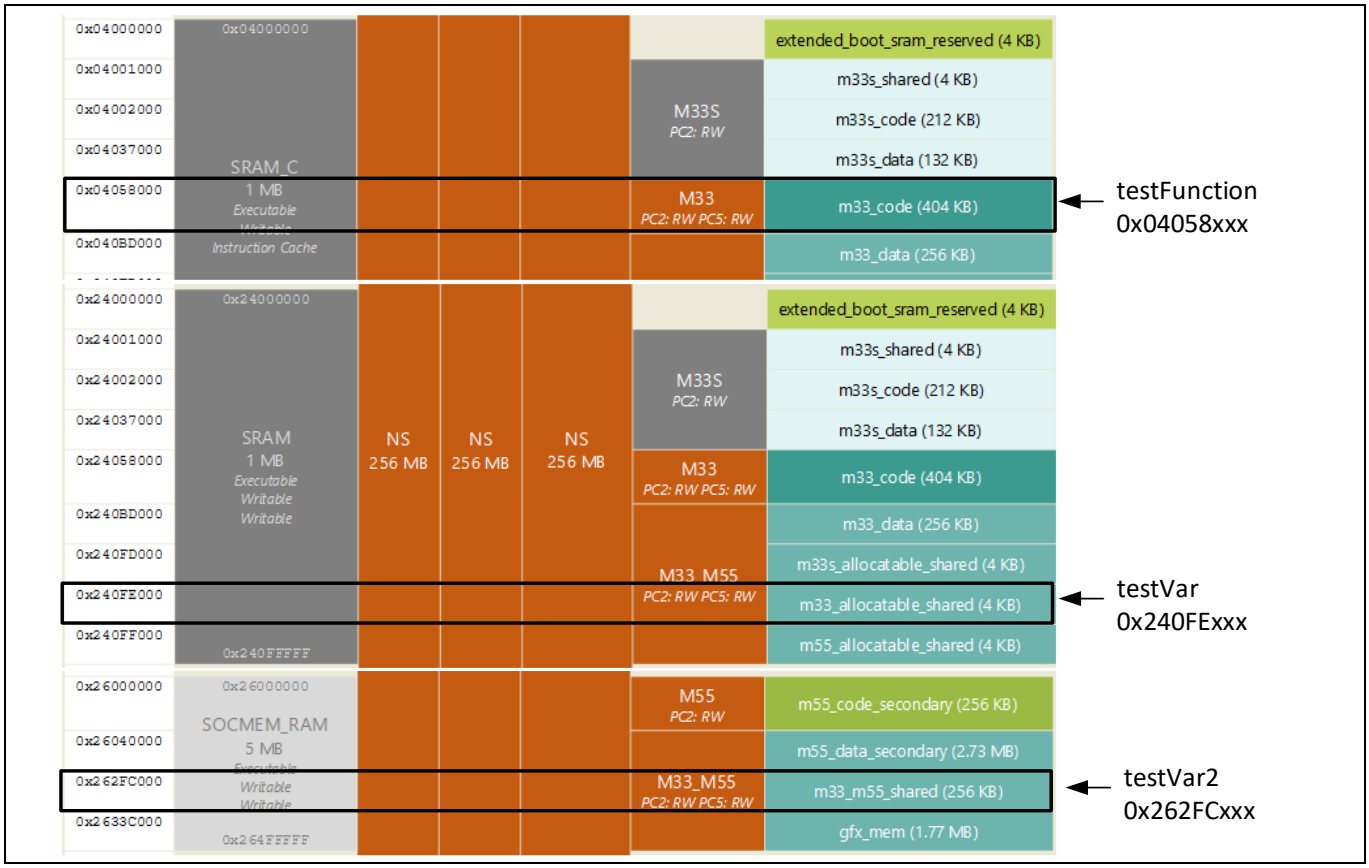


Figure 4 Updated memory locations in Memory Configurator

4.4.3 Part 3: Placing code and data in a custom region

The previous section explained how to place code and data in existing regions; however, sometimes it's useful to create new sections to customise the memory map.

This section explains how to create custom regions for variables and functions.

1. Open the **Memory Configurator** in **Device Configurator** from the Quick Panel
2. Go to the **Memory Regions** tab and perform the following modifications
   a. Expand the SRAM regions

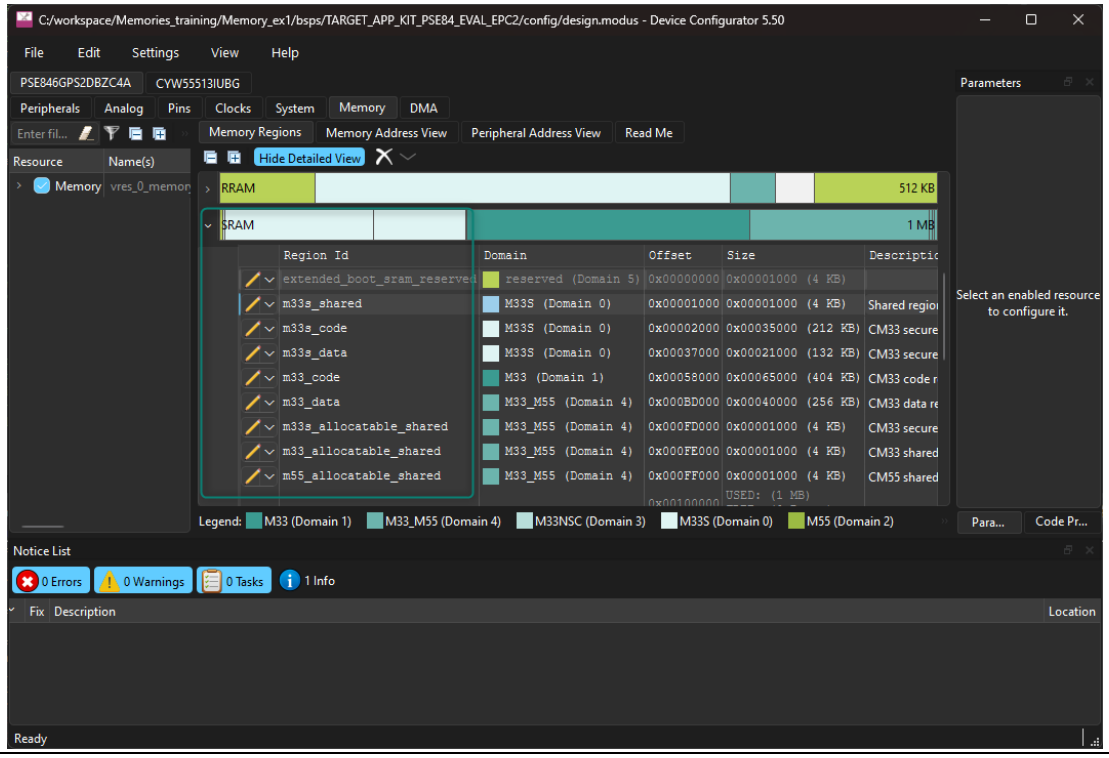


Figure 5 SRAM regions

b. Double-click the **m33_data** section and modify **Size** from 0x40000 to 0x30000. This will free a 64 kB section for a custom section

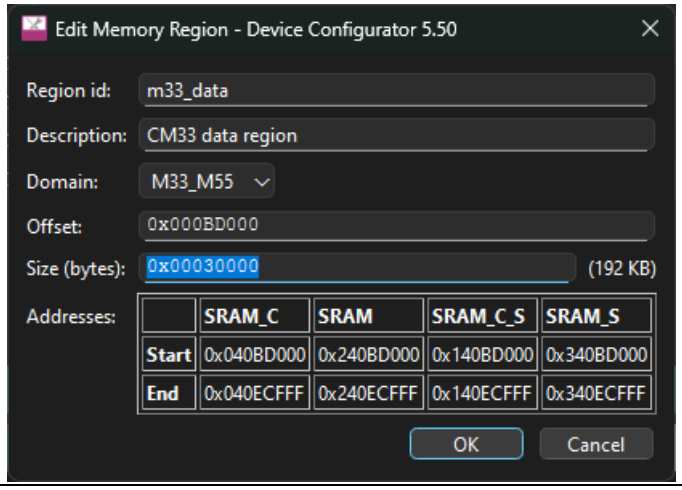


Figure 6 Modify the size of m33_data

c. Click the unallocated 64 KB region, then click **Add region**

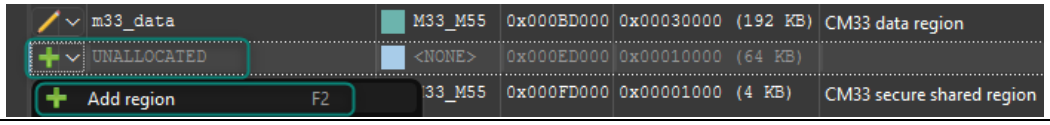


Figure 7 Add a region in unallocated region

d. Rename the **Region id** to “myData” and place it in the M33 **domain.** The section will be created right after “m33_data” and will have a size of 64 KB. Click **OK** to create the new region

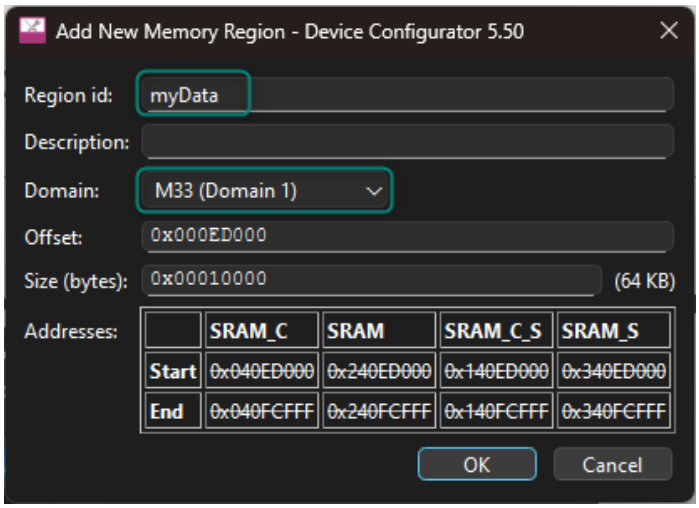


Figure 8 Add custom myData region

e. Expand RRAM regions

f. Reduce size of *user_programmable* from 0x4A000 to 0x3A000

g. Add a new “myCode” RRAM region for M33 domain in the newly unallocated 64 KB region

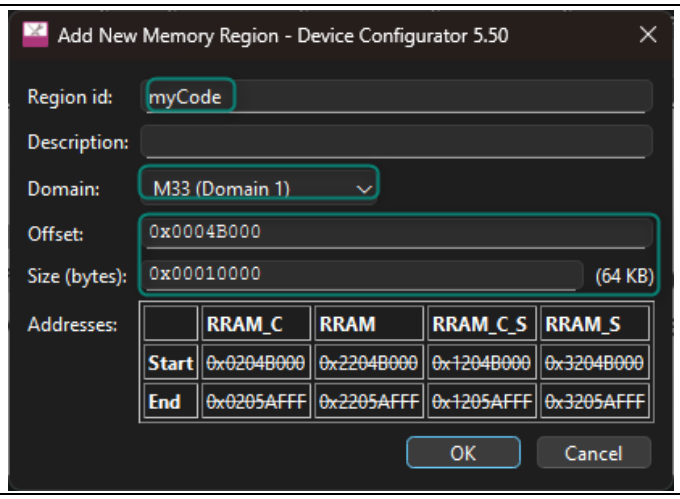


Figure 9 Add a custom myCode region

Assigning code and data to custom linker sections

h. The memory regions should look like this after these modifications

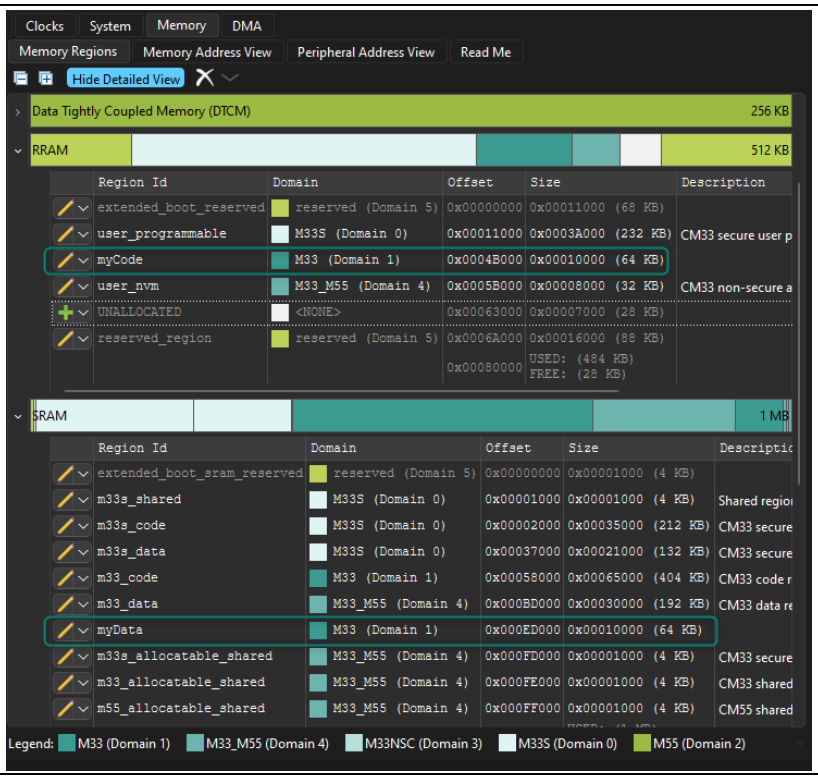


Figure 10 New sections in the Memory Configurator

i. Observe the new sections and addresses in the **Memory Address View**

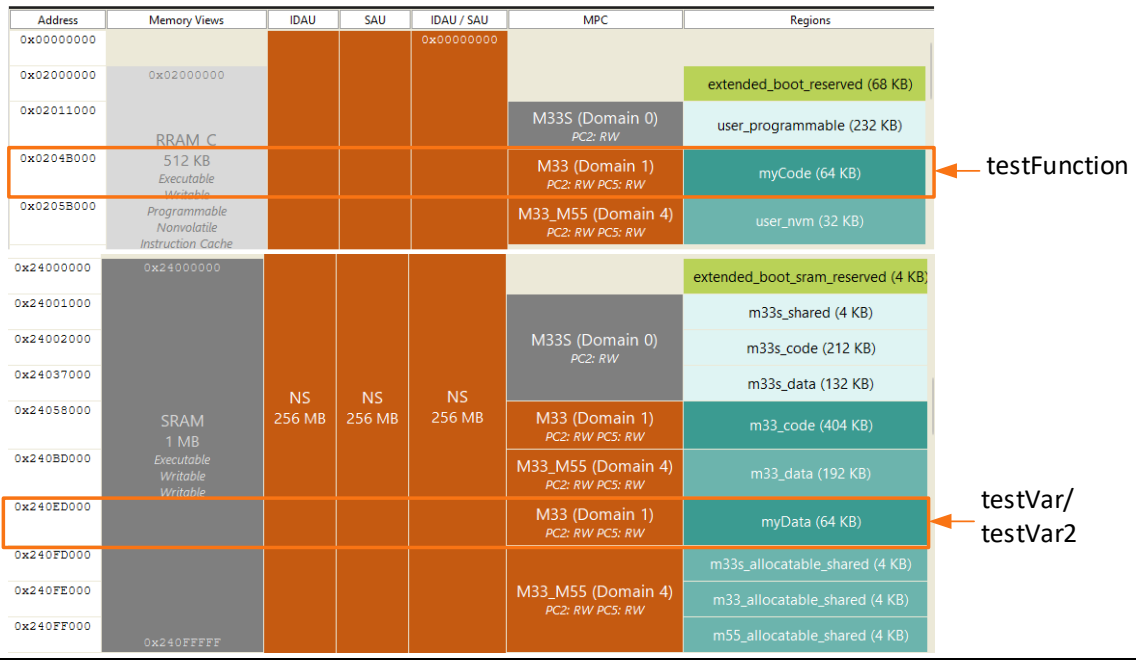


Figure 11 New sections in the memory configurator

Assigning code and data to custom linker sections

j. Save the configuration to regenerate files (Ctrl+S)

3. Add the new sections to the cm33_ns linker file:
   <example>\bsps\TARGET_APP_KIT_PSE84_EVAL_EPC2\COMPONENT_CM33\TOOLCHAIN_GCC_ARM\pse84_ns_cm33.ld

Code listing 5 Add new sections to the cm33_ns linker file

```
/* Define the sections */
SECTIONS
{
    /* New NVM section */
    .mycode : ALIGN(4)
    {
        KEEP(*(.mycode))
    } > myCode

     /* New RAM section */
    .mydata(NOLOAD) : ALIGN(4)
    {
        KEEP(*(.mydata))
    } > myData

    /* This section reserves a space for MCUBoot header */
```

4. Place the variables and function into the new sections in `proj_cm33_ns/main.c`

Code listing 6 Placing variables and functions into custom sections

```
/*******************************************************************************
* Global variables
*******************************************************************************/
/* Volatile variable in RAM */
CY_SECTION(".mydata") __attribute__((used))  volatile uint32_t testVar = 0;
CY_SECTION(".mydata") __attribute__((used))  volatile uint32_t testVar2 = 0;

/*******************************************************************************
* Local functions
*******************************************************************************/
/* Function in non-volatile memory */
CY_SECTION(".mycode") __attribute__((used)) void testFunction(void)
{
    testVar++;
    testVar2++;
}
```

5. Build and reprogram the application
6. Observe the results in a terminal, and/or in the .map file

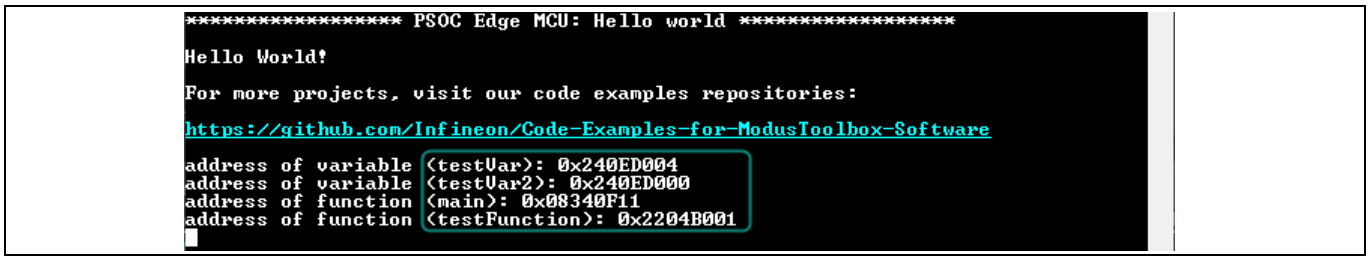


Figure 12 Console output using custom sections

4.4.4 Other things to try

Try placing variables and functions in other sections.

Note: A similar process applies for the CM33 secure and CM55 applications. Memory Configurator can define sections that apply only to a specific core or can be shared across multiple cores.

4.4.5 Key takeaways

To place functions in SRAM:

- **Time-critical code:** If a function is time-critical, meaning it needs to execute quickly and predictably, placing it in SRAM can be beneficial. SRAM is generally faster than flash memory, which is typically used to store program code. By locating time-critical functions in SRAM, you can reduce the latency associated with fetching instructions from external flash memory
- **Frequently executed code:** If a function is called repeatedly, it's a good candidate for placement in SRAM. This is because SRAM access is typically faster than external flash memory access, so you can reduce the overall execution time by minimising the number of times the function needs to be fetched from flash memory

4.5 Conclusion

In this lab, we demonstrated how to assign functions and variables to specific linker sections in a PSOC™ Edge MCU-based project, showcasing our control over memory layout and optimisation. By modifying the **PSOC™ Edge HAL Hello World** code example, we learned how to allocate variables and functions to different sections in the linker, including creating custom ones. With this knowledge, we can now apply linker section assignments to other projects, streamlining the process and gaining greater control over the memory layout of our applications.

5 Assigning a source file to a linker section

5.1 Objective

The previous lab showed how to assign specific functions or variables to memory sections. We will now show how to assign source files to a memory region, which is a more convenient approach when multiple functions or variables need to be allocated.

This lab will also show how to leverage ITCM for the CM55 core.

5.2 Project creation

For this lab, we will use the same example created in Assigning code and data to linker sections.

5.3 Description

When dealing with large codebases, manually assigning individual variables and functions to specific linker sections can be tedious and time-consuming. To overcome this limitation, you can use the file-level management option provided by compilers, which enables you to relocate entire code modules at once. By doing so, you can move all functions or data from a source file into separate sections, streamlining the process and gaining greater control over the memory layout of your application.

In this lab, you will learn how to effectively allocate CM55 code from a specific source file into a dedicated linker section in ITCM, which provides zero-latency code execution for the CM55 core. This will help us achieve a more manageable approach to memory allocation in your PSOC™ Edge MCU-based project.

5.4 Assigning files to linker sections

5.4.1 Update proj_cm55 to observe placement of code and data

1. The default CM55 application does not use retarget_io, which is a library used to send messages to the UART terminal. Add the library by following the next steps
   a. In the Quick Panel, click **Library Manager**

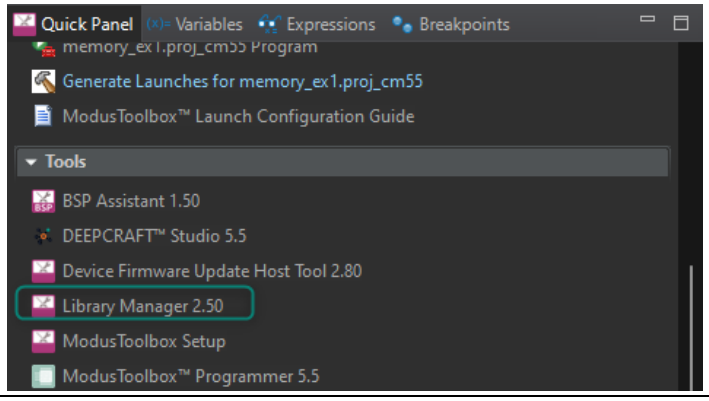


Figure 13 Open Library Manager

Assigning a source file to a linker section

b. Click **Add Library**

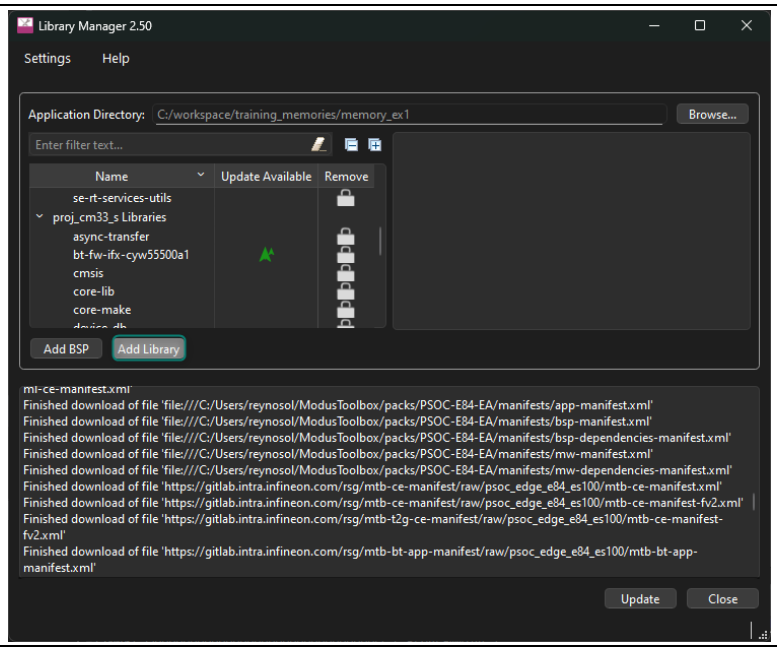


Figure 14 Adding a library

c. Select the proj_cm55 project, look for the **retarget-io** library and add it by checking the box, and click OK

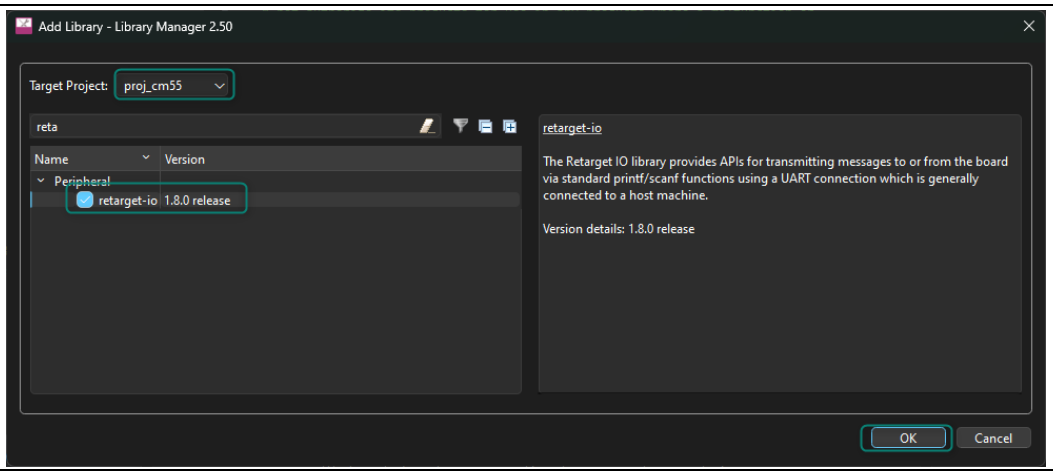


Figure 15 Add retarget-io to proj_cm55

Assigning a source file to a linker section

d. Click on **Update** and wait for completion, then **close** the Library manager

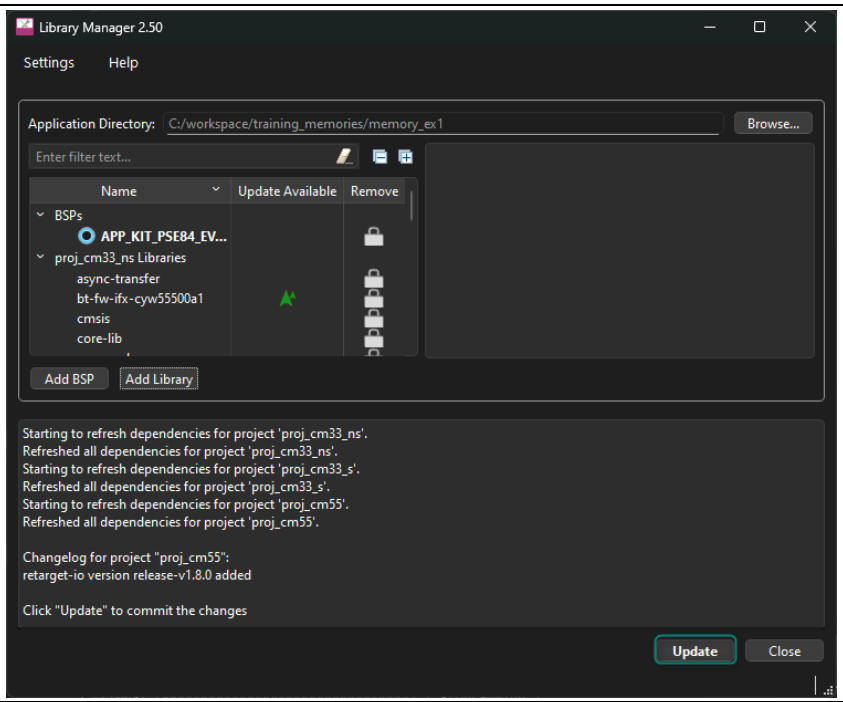


Figure 16 Update libraries

2. In the **Project Explorer**, copy-paste **retarget_io_init.c/h** from proj_cm33_ns to proj_cm55

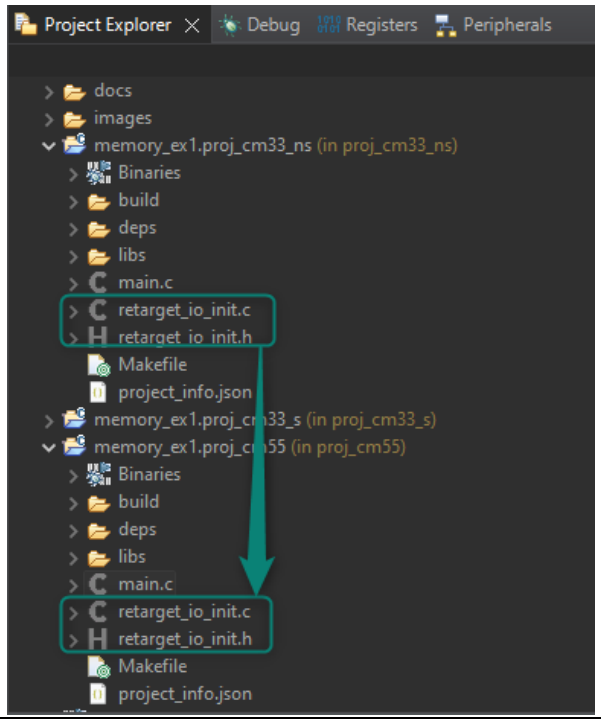


Figure 17 Copy retarget_io_init files to proj_cm55

Assigning a source file to a linker section

3. Add the following code to `proj_cm55/main.c` to initialise the retarget_io console and print the location of custom variables and functions

Code listing 7 Modification #1 to `proj_cm55/main.c`

```
/*******************************************************************************
* Header File
*******************************************************************************/

#include "cybsp.h"
#include "retarget_io_init.h"

/*******************************************************************************
* Global variables
*******************************************************************************/
/* Volatile variable in RAM */
__attribute__((used))  volatile uint32_t m55testVar = 0;

/*******************************************************************************
* Local functions
*******************************************************************************/
/* Function in non-volatile memory */
__attribute__((used)) void m55testFunction(void)
{
    m55testVar++;
}
```

Code listing 8 Modification #2 to `proj_cm55/main.c`

```
    /* Infinite loop */
    while(true);
}

init_retarget_io();

/* Enable global interrupts. */
__enable_irq();

printf("****CM55 application****\n");
printf("address of variable (m55testVar): 0x%08X\n", (unsigned int) &m55testVar);
printf("address of function (main): 0x%08X\n", (unsigned int)  main);
printf("address of function (m55testFunction): 0x%08X\n", (unsigned int)
m55testFunction);
```

Assigning a source file to a linker section

4. Modify `proj_cm33_ns/main.c` to finish printing before calling m55, to avoid conflicts using retarget-io and UART

Code listing 9 Modification #1 to `proj_cm33_ns/main.c`

```
/* App boot address for CM55 project */
#define CM55_APP_BOOT_ADDR          (CYMEM_CM33_0_m55_nvm_START + \
                                     CYBSP_MCUBOOT_HEADER_SIZE)

#define WAIT_FOR_TX_COMPLETE()    while (!(Cy_SCB_UART_IsTxComplete \
                                     (CYBSP_DEBUG_UART_HW)));

/*******************************************************************************
* Global variables
*******************************************************************************/
```

Code listing 10 Modification #2 to `proj_cm33_ns/main.c`

```
printf("address of variable (testVar): 0x%08X\n", (unsigned int) &testVar);
printf("address of variable (testVar2): 0x%08X\n", (unsigned int)
&testVar2);
printf("address of function (main): 0x%08X\n", (unsigned int)  main);
printf("address of function (testFunction): 0x%08X\n", (unsigned int)
testFunction);

WAIT_FOR_TX_COMPLETE();

/* Enable CM55. */
/* CM55_APP_BOOT_ADDR must be updated if CM55 memory layout is changed.*/
Cy_SysEnableCM55(MXCM55, CM55_APP_BOOT_ADDR, CM55_BOOT_WAIT_TIME_USEC);
```

5. Build and run the code to observe the default placement of CM55 code and data

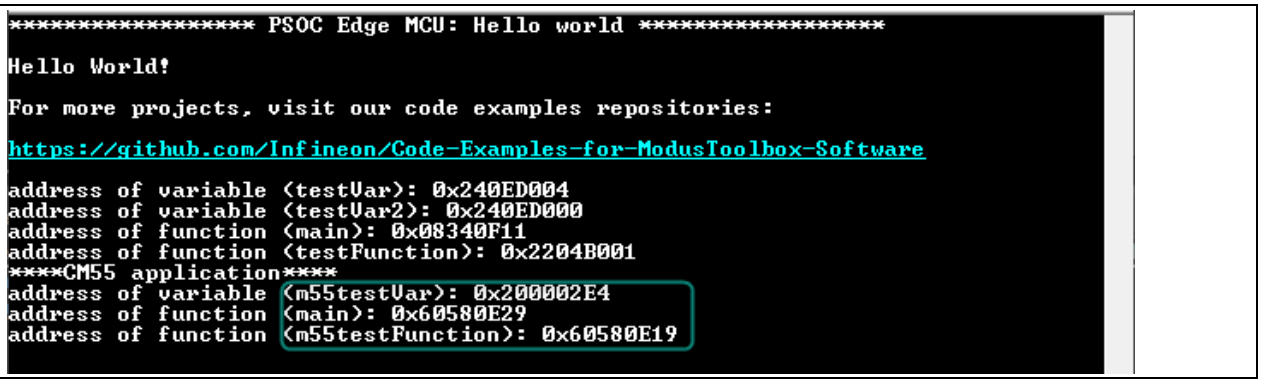


Figure 18 Default placement of code and data in the CM55 project

Table 5 Default placement of variables and functions in CM55 project

Variable/Function	Default memory location	Memory configurator region Id
m55testVar	DTCM	m55_data
m55testFunction	External flash region	m55_nvm

5.4.2 Moving proj_cm55/main.c code

1. Open the CM55 linker file located at:

<example>\bsps\TARGET_APP_KIT_PSE84_EVAL_EPC2\COMPONENT_CM55\TOOLCHAIN_GCC_ARM\pse84_ns_cm55.ld

2. Exclude the `main.c` code from the default code section "*m55_nvm_sel*" in external flash by using the EXCLUDE_FILE command

Code listing 11 Exclude main.c from external flash placement

```
/* This section is intended to hold the main application code for the Cortex-M55 */
/* Performance-sensitive or critical functions that need to be executed in sram are manually excluded */
.app_code_main : ALIGN(VECTORS_ALIGNMENT)
{
    __text_begin = .;
    __Vectors = . ;
    /* Keep the vector table */
    KEEP(*(.vectors))

    . = ALIGN(4);

    __Vectors_End = .;

    /* Keep initializers */
    KEEP(*(.init))
```

Assigning a source file to a linker section

```
    /* Keep finalisers */
    KEEP(*(.fini))
    /* Static constructors */
    *crtbegin.*(.ctors)
    *crtbegin?.*(.ctors)
    *(EXCLUDE_FILE(*crtend?.* *crtend.*) .ctors)
    *(SORT(.ctors.*))
    *(.ctors)
    /* Static destructors and atexit() */
    *crtbegin.*(.dtors)
    *crtbegin?.*(.dtors)
    *(EXCLUDE_FILE(*crtend?.* *crtend.*) .dtors)
    *(SORT(.dtors.*))
    *(.dtors)
    /* Read only code (constants) */
    *(.rodata .rodata.* .constdata .constdata.* .conststring .conststring.*)
    /* Exclude certain files from this section as they will be placed in RAM */
    *(EXCLUDE_FILE(*main.* *cy_syslib.* *cy_syslib_ext.* *cy_smif.*
*cy_smif_memnum.* *cy_smif_memslot.* *cy_smif_sfdp.* *cy_smif_hb_flash.*
*mtb_hal_memoryspi.* *mtb_serial_memory.* *freertos/Source*.* *cy_clib_support*.*
*cy_mutex_pool*.* *cy_time.* *cyabs_*.* *cy_worker_thread.* *cmsis*.*) .text*)


    . = ALIGN(4);


    __text_end = .;
} > m55_nvm_sel
```

3. Add the `main.c` file to "*m55_code_INTERNAL*" which is placed in external memory but executed from ITCM

Code listing 12 Include main.c to execute from ITCM

```
/* A section for performance-sensitive or critical functions that need to be
executed in ITCM */
.app_code_itcm : ALIGN(16)
{
    /* Reserving the first 16 bytes to avoid debug symbols (that have default
address 0x0) to be confused with actual code  */
    . += 16;
    KEEP(*(.cy_itcm))
    KEEP(*(.cy_sram_code))
    KEEP(*(.cy_ramfunc))
    KEEP(*(.text.cy_os_common))
    *cy_syslib_ext.*(.text*)
    *cy_syslib.*(.text*)
```

Assigning a source file to a linker section

```
    *cy_smif.*(.text*)
    *cy_smif_memnum.*(.text*)
    *cy_smif_memslot.*(.text*)
    *cy_smif_sfdp.*(.text*)
    *cy_smif_hb_flash.*(.text*)
    *mtb_hal_memoryspi.*(.text*)
    *mtb_serial_memory.*(.text*)
    *freertos/Source*.*(.text*)
    *cy_clib_support*.*(.text*)
    *cy_mutex_pool*.*(.text*)
    *cy_time.*(.text*)
    *cyabs_*.*(.text*)
    *cy_worker_thread.*(.text*)
    *cmsis*.*(.text*)
    *main.*(.text*)

    . = ALIGN(4);
} > m55_code_INTERNAL AT > m55_nvm_sel
```

4. Build and reprogram the application
5. Observe the new locations of main and m55testFunction, which correspond to ITCM

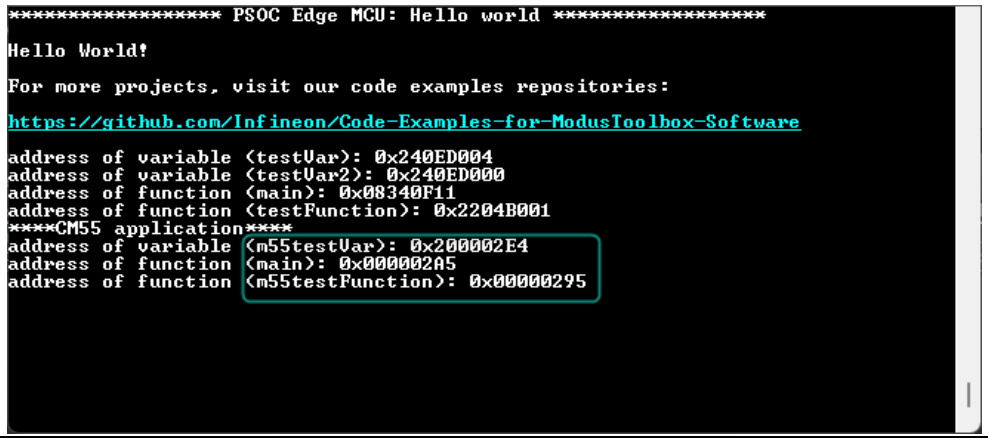


Figure 19 Terminal output after moving M55 main.c to ITCM

5.4.3 Other things to try

Try moving the other files from the PDL library, such as `cy_syspm_v4.c`, to a different code section and understand the relationship between the section and which memory they are linked.

5.4.4 Key takeaways

1. When to use ITCM and DTCM?

In CM55-based applications, consider using ITCM (Instruction tightly coupled memory) and DTCM (Data tightly coupled memory) in the following situations:

- ITCM
  - **Time-critical code:** Place time-critical code, such as interrupt handlers or low-latency functions, in ITCM to minimise latency and ensure predictable execution
  - **Frequently executed code:** Store frequently executed code, such as loops or performance-critical functions, in ITCM to reduce the latency associated with instruction fetches
- DTCM
  - **Frequently accessed data**: Place frequently accessed data, such as stack variables or performance-critical data structures, in DTCM to reduce the latency associated with data accesses
  - **Low-latency data access:** Use DTCM for data that requires low-latency access, such as in real-time systems or applications with strict timing constraints

By using ITCM and DTCM, you can optimise the performance and responsiveness of your CM55-based application by reducing memory access latency and improving instruction and data fetch times.

2. Which files to place entirely in SRAM or TCM?

- **Low-power applications:** In low-power applications, reducing the number of external flash memory accesses can help minimize power consumption. By placing functions in "*cy_syspm_v4.c*" in SRAM, you can reduce the power consumption associated with external flash memory accesses
- **Intensive processing:** computationally intensive algorithms can benefit from executing from low-latency memories

5.5 Conclusion

In this lab, we demonstrated how to assign code from files into specific linker sections, which is a more convenient way to customize the memory allocation of complete files.

We also demonstrated the ease of placing functions in ITCM, which provides faster execution using the CM55 core.

These features can help reduce power consumption and improve the performance of your application.

6 Placing and executing code from RRAM

6.1 Objective

In this exercise, you will learn how to move the application images into and out of RRAM.

The PSOC™ Edge MCU utilises an extended boot program to verify the images and jump to the appropriate location to begin the CM33_S application. To do this, a region of RRAM must be reserved for extended boot. A second region at the end of RRAM must also be reserved for extended boot. The M33_S, M33_NS, and M55 images can all be moved into RRAM so that the SMIF interface is not needed.

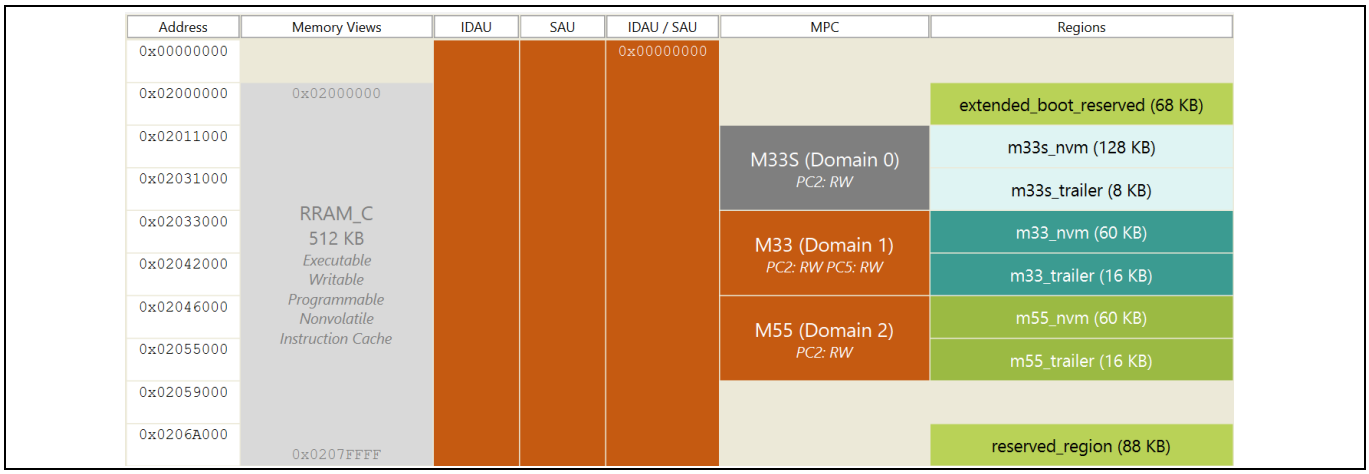


Figure 20 Memory map of all images in RRAM

A common use case for the PSOC™ Edge is to use the CM33 CPU to set up the peripherals and run the low-power operations, then use the CM55 and other components on PD1 for more intensive processing and high-speed items. However, the CM55 application is much larger and must be stored in an external flash. Let's keep the CM33_S and CM33_NS application images in RRAM and move the CM55 application image back into external flash.

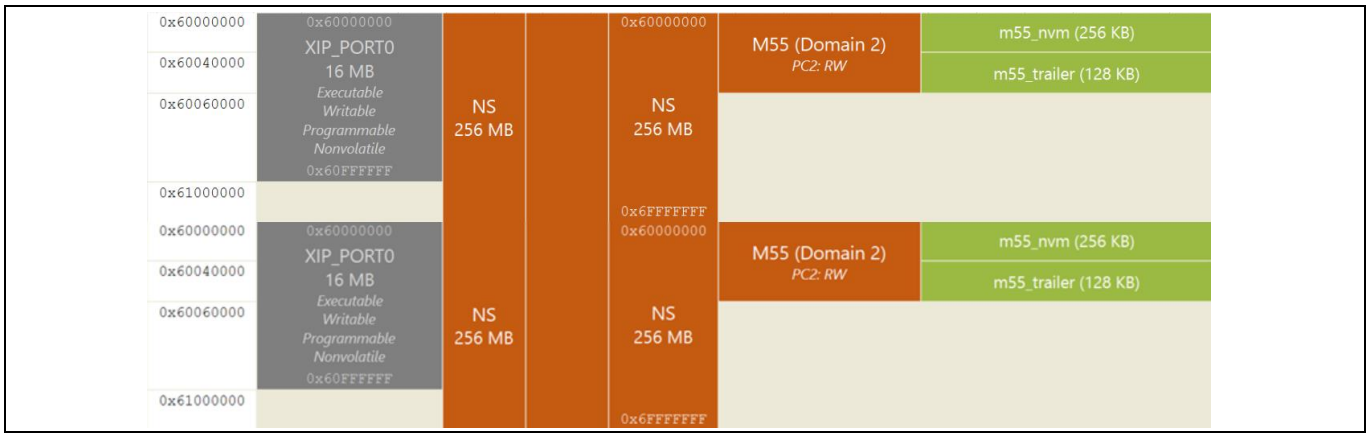


Figure 21 Memory map of the CM33 images in RRAM and the CM55 image in external flash

Placing and executing code from RRAM

The RRAM memory in the PSOC™ Edge MCU is divided into two main regions:

- One-time programmable (OTP)
- Non-volatile memory (NVM) (512 KB)
- MAIN NVM (user programmable and extended BOOT) (256 KB)
- PROTECTED NVM (only accessible by Secured Enclave) (256 KB)

The MAIN NVM region is user-programmable and contains an extended BOOT area, whereas the PROTECTED NVM region is only accessible by the Secured Enclave, providing an additional layer of security. See the PSOC™ Edge MCU datasheet for the actual size of the NVM and OTP region.

When it comes to storing code and data for CM33 and CM55, it's essential to understand the properties and limitations of RRAM, a non-volatile memory technology that retains its stored data even when power is turned off.

However, RRAM is a scarce resource and must be used judiciously, requiring developers and designers to carefully consider the trade-offs of using it for storing CM33 and CM55 code and data. This means weighing the benefits of RRAM's non-volatility and potential power consumption reductions against its limited capacity, and considering alternative storage options to ensure an efficient system design.

6.2 Description

By default, the three projects are in the external flash connected to SMIF0. However, in this exercise, you will modify the memory allocation to execute all three projects in the RRAM of the PSOC™ Edge device.

You will use a basic code example, such as the PSOC™ Edge MCU: Hello world; customise the memory allocation to reserve the required RRAM for extended boot and system usage; and then allocate 128 KB of RRAM for the CM33_S project and 60 KB of RRAM for each of the CM33_NS and CM55 projects.

Then you will put the CM55 application image back into the external flash and make the necessary modifications to the CM33_S and CM33_NS projects to launch the CM55 application from the external flash.

6.3 Project creation

Follow steps 1-6 in Appendix A: Creating a PSOC™ Edge application in ModusToolbox™ to create a new application using the Hello World example. Use a descriptive project name like **pse_memory_ex2**.

6.4 Move all application images into RRAM

6.4.1 Changes in main.c file of the *cm33_ns* project

To verify the allocation of code, add a printf statement to print the values of the main function to the terminal. This allows you to confirm that the code and data have been successfully relocated to the desired linker sections.

Code listing 13 Add a printf statement for the main function location

```
printf("Hello World!\r\n\n");
printf("For more projects, "
       "visit our code examples repositories:\r\n\n");

printf("https://github.com/Infineon/"
       "Code-Examples-for-ModusToolbox-Software\r\n\n");
printf("address of function            (main): %p\n", main);

/* Enable CM55. */
/* CM55_APP_BOOT_ADDR must be updated if CM55 memory layout is changed.*/
```

6.4.2 Memory personality update

The default memory configuration places all application code for the CM33_S, CM33_NS, and CM55 cores in external flash memory, accessible through the Serial memory interface block 0 (SMI0).

In this exercise, we will move all three applications (CM33_S, CM33_NS, and CM55) from external flash to internal RRAM.

To accomplish this, we will use the memory configurator tool within the Device Configurator.

Follow these steps:

1. Launch the **Device Configurator** from the Quick Panel and open the **Memory** tab
2. Delete the “user_programmable” and “user_nvm” regions in the **RRAM** region

Placing and executing code from RRAM

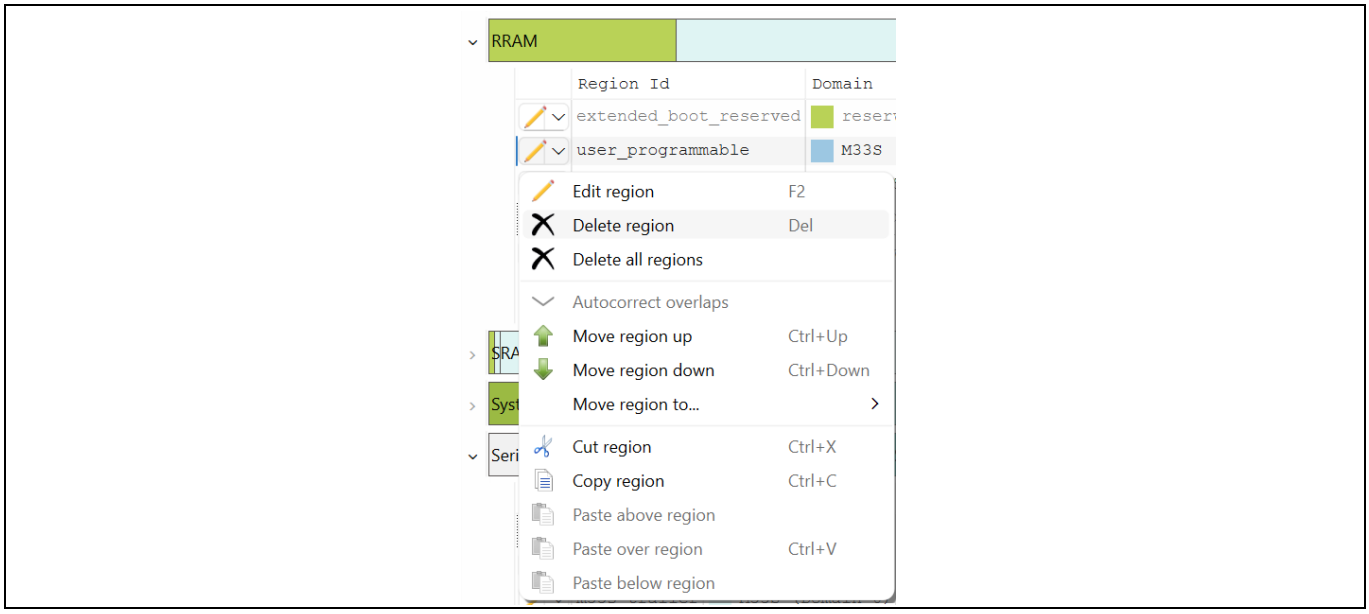


Figure 22 Deleting regions in Memory Configurator

3. Expand the **Serial memory interface block 0, memory 1** region and set the CM33_S, CM33_NS, and CM55 image and trailer sizes according to the Table 6

Table 6 Image and trailer sizes for RRAM

Region Id	Size	Size (HEX)
m33s_nvm	128 KB	0x00020000
m33s_trailer	8 KB	0x00002000
m33_nvm	60 KB	0x0000F000
m33_trailer	16 KB	0x00004000
m55_nvm	60 KB	0x0000F000
M55_trailer	16 KB	0x00004000

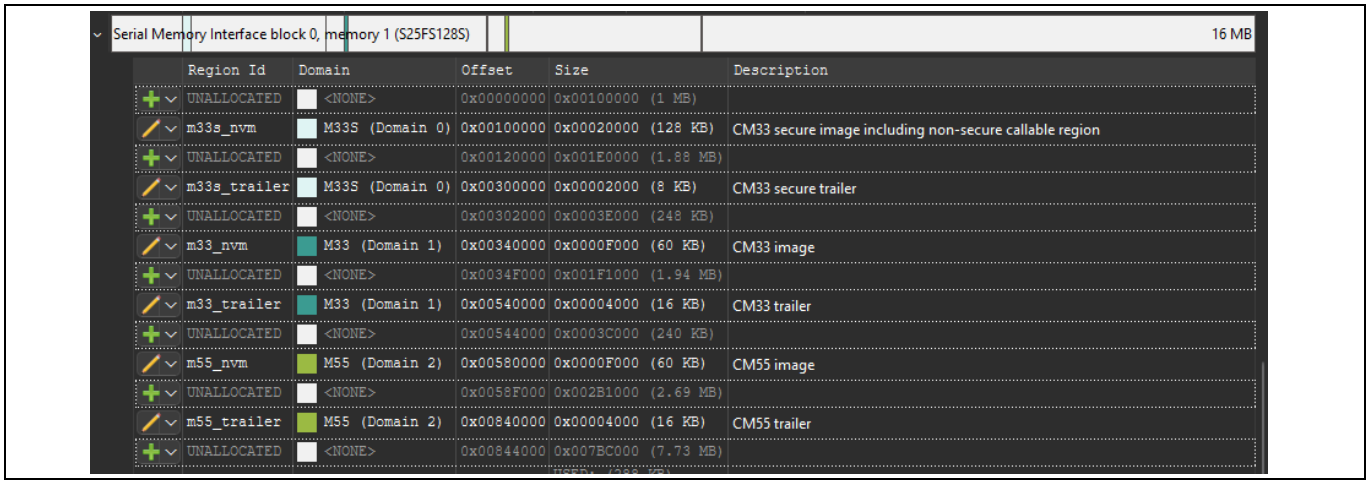
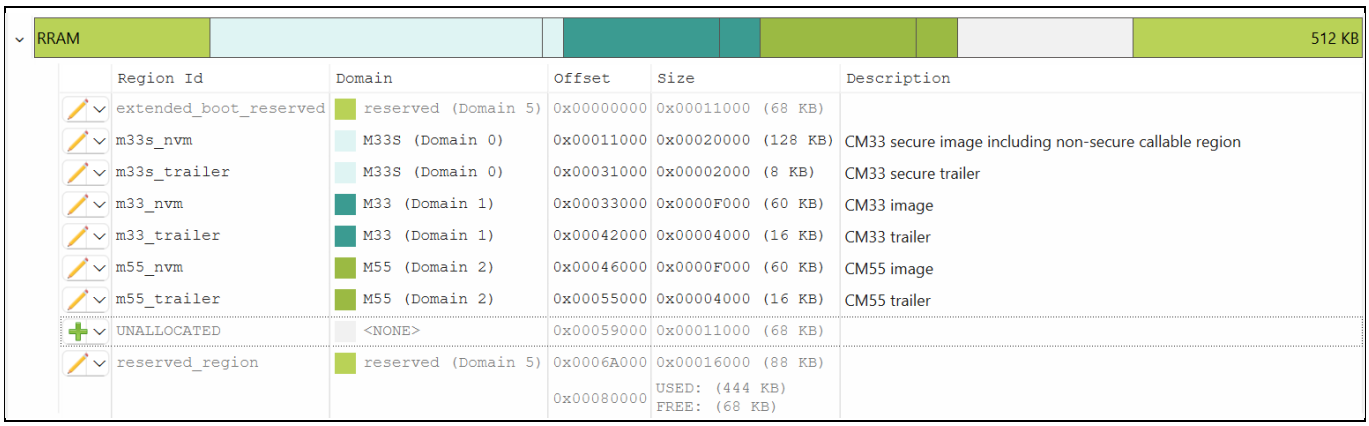


Figure 23 External memory after adjusting region sizes

4. Select m33s_nvm and cut it (Ctrl+x), then go to RRAM and select the UNALLOCATED region, and paste (Ctrl+v). Another option is to right-click and select **Move region to… RRAM**

   Note: The region should be allocated in the unallocated region after extended_boot_reserved.

5. Repeat to move all regions into RRAM



Region Id	Domain	Offset	Size	Description
extended_boot_reserved	reserved (Domain 5)	0x00000000	0x00011000 (68 KB)	
m33s_nvm	M33S (Domain 0)	0x00011000	0x00020000 (128 KB)	CM33 secure image including non-secure callable region
m33s_trailer	M33S (Domain 0)	0x00031000	0x00002000 (8 KB)	CM33 secure trailer
m33_nvm	M33 (Domain 1)	0x00033000	0x0000F000 (60 KB)	CM33 image
m33_trailer	M33 (Domain 1)	0x00042000	0x00004000 (16 KB)	CM33 trailer
m55_nvm	M55 (Domain 2)	0x00046000	0x0000F000 (60 KB)	CM55 image
m55_trailer	M55 (Domain 2)	0x00055000	0x00004000 (16 KB)	CM55 trailer
UNALLOCATED	<NONE>	0x00059000	0x00011000 (68 KB)	
reserved_region	reserved (Domain 5)	0x0006A000	0x00016000 (88 KB)	
		0x00080000	USED: (444 KB) FREE: (68 KB)	

Figure 24 Move all images and trailers from external memory to RAM

6. Save design.modus to regenerate files, then close Device Configurator

6.4.3 Update CM55 App start address

1. Update the CM55 app start location in the `cm33_ns/main.c` file

Code listing 14 Update the CM55 app start location

```
/* App boot address for CM55 project */
#define CM55_APP_BOOT_ADDR          (CYMEM_CM33_0_m55_nvm_C_START + \
                                     CYBSP_MCUBOOT_HEADER_SIZE)
```

6.4.4 Modify “boot_with_extended_boot.json”

ModusToolbox™ utilises JSON files to handle application code signing and merging.

1. Open the JSON file located at: *<example>\configs\boot_with_extended_boot.json*
2. Modify as shown
   a. Adjust the size of CM33_S section and trailer

Code listing 15 Adjust CM33_S in JSON file

```
"name": "metadata_proj_cm33_s",
"description": "add MCUboot metadata to the hex",
"enabled" : true,
"commands" :
[
    {
        "command" : "sign",
        "inputs" :
        [
            {
                "file" : "../build/project_hex/proj_cm33_s.hex",
                "header-size": "0x400",
                "fill-value" : "0x00",
                "slot-size" : "0x220000",
                "hex-address" : "{{CYMEM_CM33_0_S_m33s_nvm_S_START}}"
            }
        ],
```

   b. Add the cm55 section

Code listing 16 Add CM55 sections to JSON file

```
                "extra_config":
                [
                    {
                        "project": "proj_cm33_ns",
                        "debug_config_name" : "proj_cm33_ns",
                        "default":false,
                        "build_dependency" : "project"
                    }
                ]
            }
        ]
    },
   {
```

```
        "name": "metadata_proj_cm55",
        "enabled" : true,
        "commands" :
        [
            {
                "command" : "sign",
                "inputs" :
                [
                    {
                        "file" : "../build/project_hex/proj_cm55.hex",
                        "header-size": "0x400",
                        "fill-value" : "0x00",
                        "slot-size" : "0x13000",
                        "hex-address" : "{{CYMEM_CM33_0_m55_nvm_START}}"
                    }
                ],
                "outputs":
                [
                    {
                        "file" : "../build/project_hex/proj_cm55_signed.hex",
                        "format": "ihex"
                    }
                ]
            }
        ]
    },

    {
        "name": "merge",
```

c. Add the cm55 image to the combined hex file

Code listing 17 Add cm55 image to combined hex file

```
"name": "merge",
"description": "combine individual project hex files into a single hex",
"enabled": true,
"commands" :
[
    {
        "command" : "merge",
        "inputs" :
        [
            {
```

```
                "file" : "../build/project_hex/proj_cm33_s_signed.hex"
            },
            {
                "file" : "../build/project_hex/proj_cm33_ns_shifted.hex"
            },
            {
                "file" : "../build/project_hex/proj_cm55_signed.hex"
            }
        ],
```

6.4.5 Hardware (Boot switch) setting

Most code examples for PSOC™ Edge execute out of external flash by default, including the Hello World example used for previous labs; however, for this lab, the application should execute from RRAM.

To launch CM33S from RRAM, you need to ensure that BOOT_SWITCH (BOOT_SW/SW6) P6.0 is set to LOW (0). See Appendix B: KIT_PSE84_EVAL details to observe the location of the boot switch.

6.4.6 Build load and run

Build, program, and observe the output that prints the address of the modified variables and functions.

```
****************** PSOC Edge MCU: Hello world ******************

Hello World!

For more projects, visit our code examples repositories:

https://github.com/Infineon/Code-Examples-for-ModusToolbox-Software

address of function           (main): 0x2033ea9
```

Figure 25 Terminal output with all images in RRAM

6.4.7 Output

Table 7 shows the modified memory locations

Table 7 Modified memory locations

Variable/Function	Default memory location	Updated memory location
cm33_ns main	External flash region	RRAM code region
cm55 main	External flash region	RRAM code region

Placing and executing code from RRAM

The memory sections used for this application are shown in the memory map Figure 26.

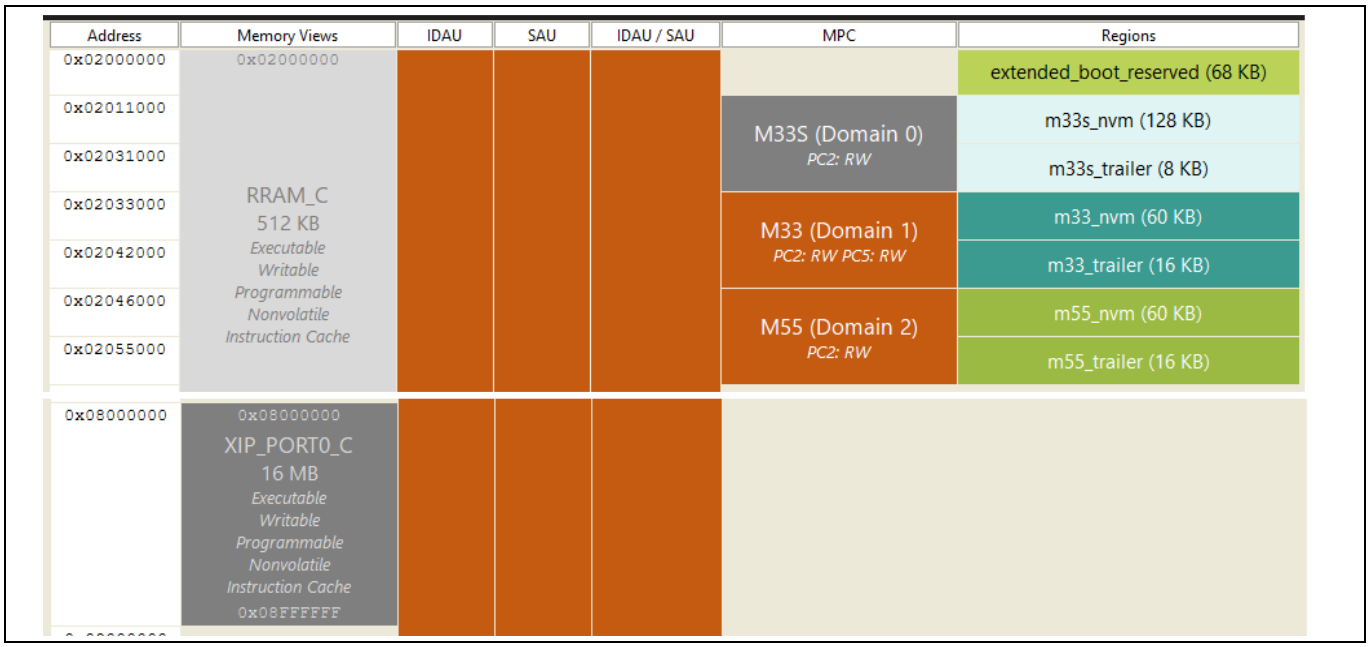


Figure 26 Updated memory map in Memory Configurator

6.5 Moving the CM55 image to the external flash

6.5.1 Observe default memory placement in the CM55 project

Before we move the CM55 image, let's print out the location of the CM55 main function.

1. Add the retarget-io library and init files to the CM55 project by following steps 1-2 from section Update proj_cm55 to observe placement of code and data
2. Add the following code to the `proj_cm55/main.c` to initialize the retarget-IO library and print out the main function location

Code listing 18 Include retarget_io_init.h

```
/*******************************************************************************
* Header File
*******************************************************************************/

#include "cybsp.h"
#include "retarget_io_init.h"
```

Placing and executing code from RRAM

Code listing 19 Initialize retarget-IO library and print location of main function

```
/* Initialize retarget-io middleware */
init_retarget_io();

/* Enable global interrupts. */
__enable_irq();

printf("****CM55 application****\n");
printf("address of function             (main): %p\n", main);

/* Put the CPU to Deep Sleep. */
```

3. Add the following code to the `proj_cm33_ns/main.c` to avoid conflicts using printf

Code listing 20 Add WAIT_FOR_TX_COMPLETE macro

```
/* App boot address for CM55 project */
#define CM55_APP_BOOT_ADDR          (CYMEM_CM33_0_m55_nvm_START + \
                                     CYBSP_MCUBOOT_HEADER_SIZE)

#define WAIT_FOR_TX_COMPLETE()    while (!(Cy_SCB_UART_IsTxComplete \
                                     (CYBSP_DEBUG_UART_HW)));
```

Code listing 21 Call macro

```
printf("address of function             (main): %p\n", main);

WAIT_FOR_TX_COMPLETE();

/* Enable CM55. */
```

4. Build, program the device, and observe the terminal output

```
****************** PSOC Edge MCU: Hello world ******************

Hello World!

For more projects, visit our code examples repositories:

https://github.com/Infineon/Code-Examples-for-ModusToolbox-Software

address of function             (main): 0x2033ea9
****CM55 application****
address of function             (main): 0x2046e11
```

Figure 27 Terminal output with all images in RRAM

6.5.2 Update the memory locations in the device configurator

1. Open the **Device Configurator** from the Quick panel and navigate to the **Memory** tab
2. Move the **m55_nvm** and **m55_trailer** from RRAM to the **Serial Memory Interface block 0**. The memory in the SMIF must be in 128 KB sizes, so the size of the image blocks will need to be updated to 256 KB (0x40000) and 128 KB (0x20000), respectively

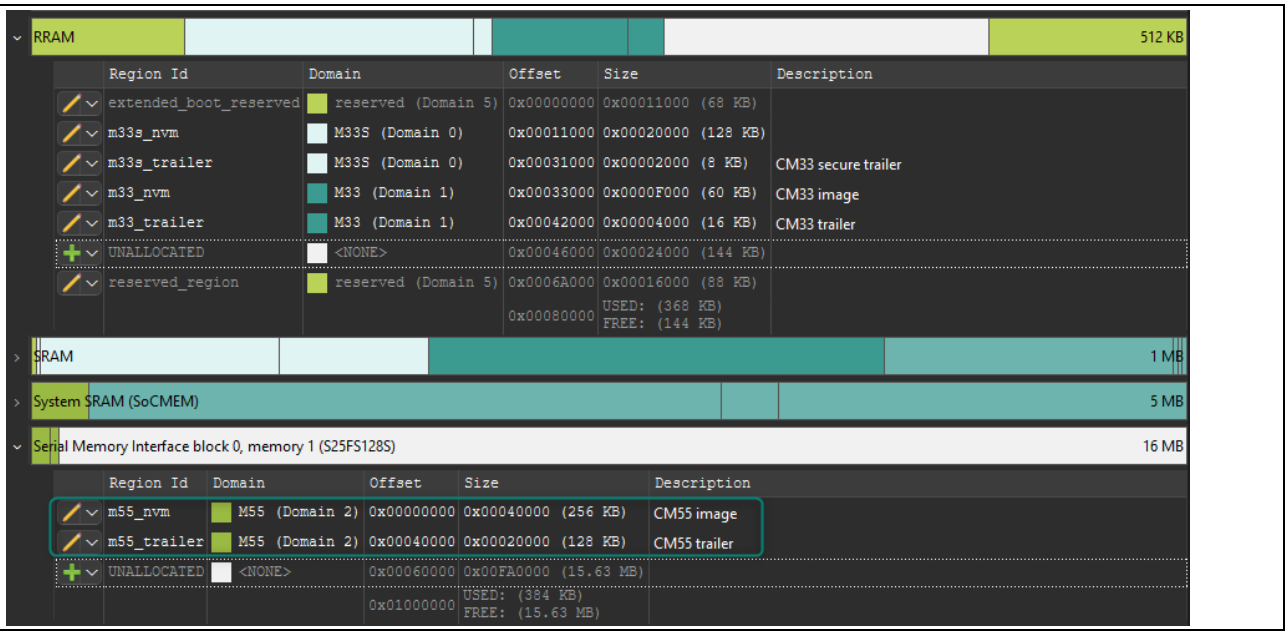


Figure 28 Updated CM55 image and trailer location

3. Save the design.modus file in the **Device Configurator** to regenerate files

6.5.3 Update CM55 App start address

1. Since the CM55 application is back to external memory, revert the CM55 app start location in the `proj_cm33_ns/main.c` file

Code listing 22 Update the CM55 app start location

```
/* App boot address for CM55 project */
#define CM55_APP_BOOT_ADDR          (CYMEM_CM33_0_m55_nvm_START + \
                                     CYBSP_MCUBOOT_HEADER_SIZE)
```

6.5.4 Configure and enable the SMIF in the CM33_S user application

1. Add defines to the *proj_cm33_s/Makefile* to skip the MPC and PPC initialization so that the SMIF can be configured in the CM33_S user code

Code listing 23 Add defines to skip MPC and PPC initialization

```
# Like SOURCES, but for include directories. Value should be paths to
# directories (without a leading -I).
INCLUDES=

DEFINES+=CYBSP_SKIP_MPC_INIT
DEFINES+=CYBSP_SKIP_PPC_INIT
```

2. Copy *external_memory.c/.h* files from the lab sources to proj_cm33_s

Source:
<lab sources>/lab3_PlacingAndExecutingCodeFromRRAM/lab3_b/proj_cm33_s/external_memory.c/h
Destination:
<example>/proj_cm33_s

3. Modify `proj_cm33_s/main.c` to initialize the SMIF, MPC, and PPC.
   This step requires several modifications. Alternatively, copy the file from the lab sources

Code listing 24 Include external_memory.h

```
/*******************************************************************************
* Header Files
*******************************************************************************/

#include "cy_pdl.h"
#include "cybsp.h"
#include "external_memory.h"
```

Code listing 25 Add a status variable in main routine

```
int main(void)
{
    uint32_t ns_stack;
    cy_cmse_funcptr NonSecure_ResetHandler;
    cy_rslt_t result;
    cy_en_smif_status_t status;
```

Placing and executing code from RRAM

Code listing 26 Initialize clocks for MMIO_TCM and SMIF0, and initialize SMIF

```
    /* Enable global interrupts */
    __enable_irq();

    /*
    * Initialize the clock for the APP_MMIO_TCM (512K) peripheral group.
    * This sets up the necessary clock and peripheral routing to ensure
    * the APP_MMIO_TCM can be correctly accessed and utilized.
    */
    Cy_SysClk_PeriGroupSlaveInit(
        CY_MMIO_CM55_TCM_512K_PERI_NR,
        CY_MMIO_CM55_TCM_512K_GROUP_NR,
        CY_MMIO_CM55_TCM_512K_SLAVE_NR,
        CY_MMIO_CM55_TCM_512K_CLK_HF_NR
    );

    /*
    * Initialize the clock for the SMIF0 peripheral group.
    * This sets up the necessary clock and peripheral routing to ensure
    * the SMIF0 can be correctly accessed and utilized.
    */
    Cy_SysClk_PeriGroupSlaveInit(
        CY_MMIO_SMIF0_PERI_NR,
        CY_MMIO_SMIF0_GROUP_NR,
        CY_MMIO_SMIF0_SLAVE_NR,
        CY_MMIO_SMIF0_CLK_HF_NR
    );

    /* Initialize SMIF in QSPI mode */
    status = external_memory_init(QSPI);
    if(CY_SMIF_SUCCESS != status)
    {
        /* Disable all interrupts. */
        __disable_irq();

        CY_ASSERT(0);

        /* Infinite loop */
        while(true);
    }
```

Placing and executing code from RRAM

Code listing 27 Initialize MPC and PPC

```
    /* Initialize MPC and PPC before executing non-secure application */

    /* Memory protection initialization */
    result = Cy_MPC_Init();
    if (CY_RSLT_SUCCESS != result)
    {
        CY_ASSERT(0);

        /* Infinite loop */
        while(true);
    }

    /* Peripheral protection initialization (PPC0) */
    result = Cy_PPC0_Init();
    if (CY_RSLT_SUCCESS != result)
    {
        CY_ASSERT(0);

        /* Infinite loop */
        while(true);
    }

    /* Peripheral protection initialization (PPC1) */
    result = Cy_PPC1_Init();
    if (CY_RSLT_SUCCESS != result)
    {
        CY_ASSERT(0);

        /* Infinite loop */
        while(true);
    }

    ns_stack = (uint32_t)(*((uint32_t*)CM33_NS_APP_BOOT_ADDR));
```

6.5.5 Modify “boot_with_extended_boot.json”

The size of the CM55 application was modified, so the corresponding JSON file needs to be updated to correctly handle the application code signing and merging.

1. Open *configs/boot_with_extended_boot.json* and modify as shown below

Code listing 28 Update the boot_with_extended_boot.json file

```
"name": "metadata_proj_cm55",
"enabled" : true,
"commands" :
[
    {
        "command" : "sign",
        "inputs" :
        [
            {
                "file" : "../build/project_hex/proj_cm55.hex",
                "header-size": "0x400",
                "fill-value" : "0x00",
                "slot-size" : "0x60000",
                "hex-address" : "{{CYMEM_CM33_0_m55_nvm_START}}"
            }
        ],
```

6.5.6 Build load and run

Build, program, and observe the output that prints the address of the modified functions.

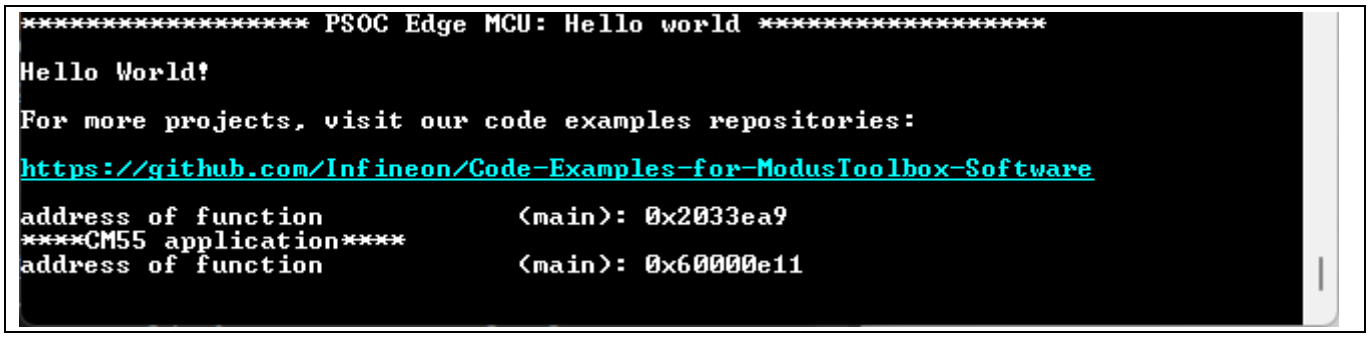


Figure 29 Terminal output with CM33 images in RRAM and CM55 image in external flash

6.5.7 Output

Table 8 shows the modified memory locations.

Table 8 Modified memory locations

Variable/Function	Previous memory location	Updated memory location
cm33_ns main	RRAM code region	RRAM code region
cm55 main	RRAM code region	External flash region

Memory sections used for this application are shown in the Figure 30.

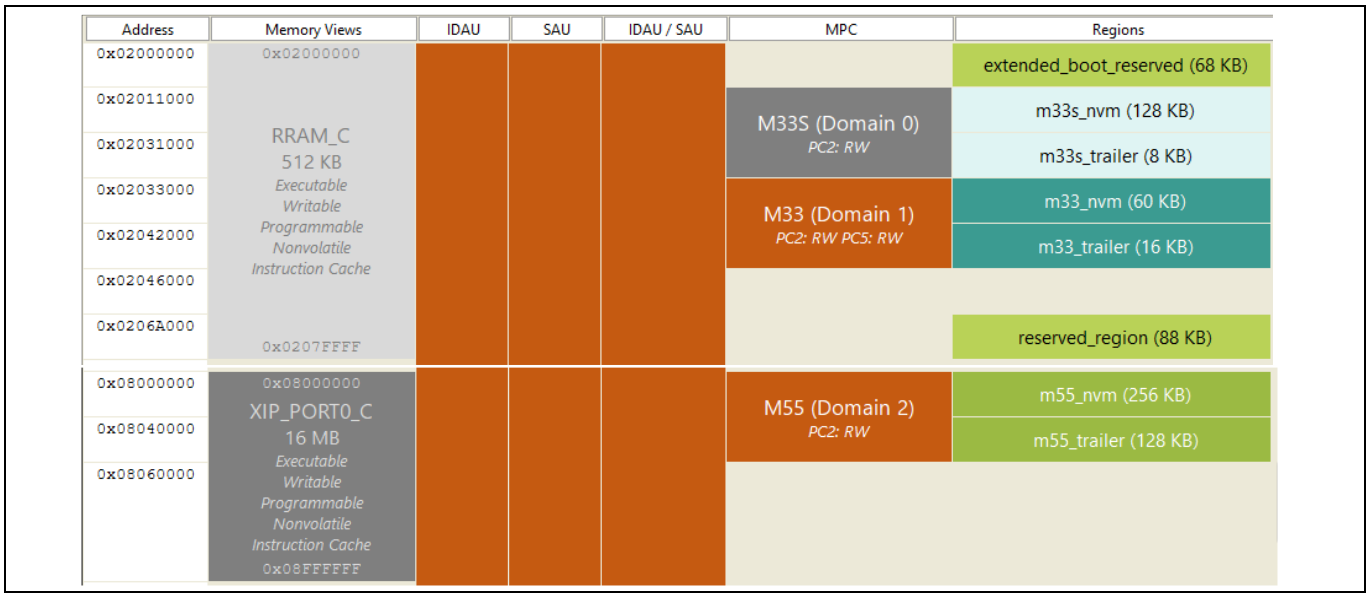


Figure 30 Updated memory map in Memory Configurator

6.6 Key takeaways

When to use RRAM for code storage?

- In low-power applications, consider placing code in Resistive random access memory (RRAM to minimise power consumption. This is particularly beneficial for power management applications, as it reduces the power consumption associated with external flash memory accesses
- Applications that require a hibernate-wake sequence can leverage RRAM to execute basic initialization tasks on external memory. In such cases, a small power management application that initializes the system and starts the main application after hibernation should be stored in RRAM
- Additionally, it can be used for data storage like EEPROM emulation

6.7 Conclusion

In this exercise, you successfully placed and executed all three projects (CM33 S, CM33 NS, and CM55) in the RRAM of a PSOC™ Edge device. Then, you moved the CM55 application image back into the external flash. These steps enable you to manage the memory locations of images based on design requirements.

7 Placing and executing code from SRAM

7.1 Objective

In this exercise, you will learn how to execute code from SRAM by modifying the default memory map to load all three projects from external flash to internal SRAM and execute in place. You will customize the memory allocation to place all three projects in SRAM, dividing the space into separate regions for each project.

7.2 Description

Most code examples included in ModusToolbox™ place and execute code from the external flash memory connected to SMIF0. This is a convenient method that provides fewer limitations on code size; however, it is not as efficient as running from internal memory in terms of power and performance. The previous lab demonstrated how to use internal RRAM, and this lab will demonstrate how to modify the memory map to place the projects in external flash, load them into SRAM, and execute them from SRAM at run-time.

The SRAM space will be divided into separate regions for each project, allocating space for the boot-loader data region, CM33S image, and CM33S data region.

Additionally, you will accommodate space in the RRAM for the Edge Protect Bootloader, which will be used to load all projects to SRAM.

7.3 Project creation

This example leverages the Edge Protect Bootloader to load the projects into RAM. We will start by creating two projects in the workspace.

1. Create an Edge Protect Bootloader example.
   a. Follow steps 1-4 in Appendix A: Creating a PSOC™ Edge application in ModusToolbox™ to create a new application
   b. When selecting the application, select the **PSOC™ Edge Protect Bootloader** under the **Security** section. Select the **checkbox** near the application and click **Create**

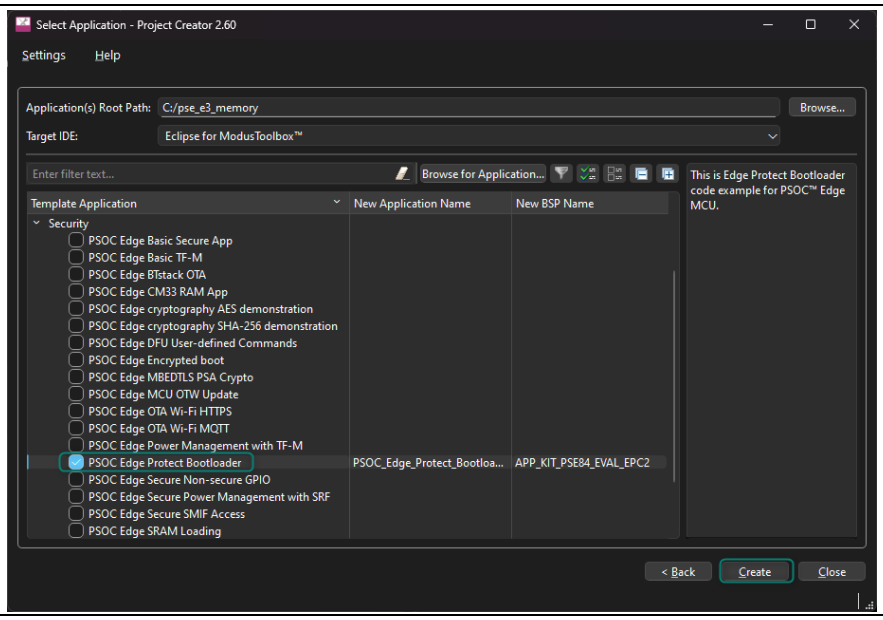


Figure 31 Selecting the PSOC™ Edge Protect Bootloader

2. Create a pse_memory_ex3 example based on Hello World
   a. Follow steps 1-6 in Appendix A: Creating a PSOC™ Edge application in ModusToolbox™ to create a new application using the Hello World example. Use a descriptive project name, such as **pse_memory_ex3**

Both examples should be placed in the Project Explorer after completing this section.

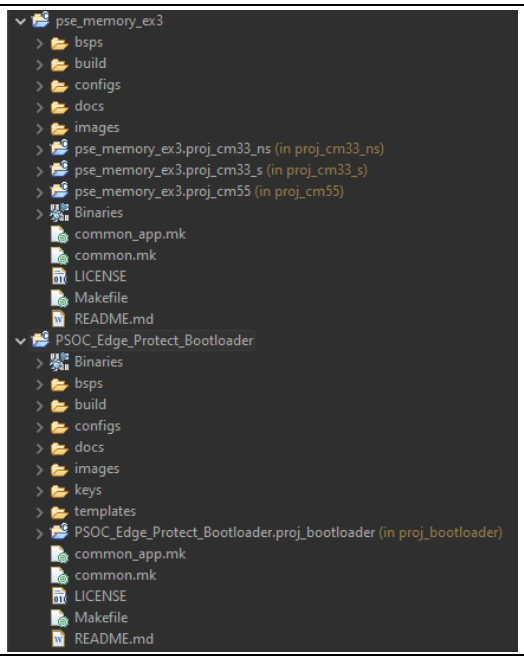


Figure 32 Project Explorer window with both projects

7.4 Using the Edge Protect Bootloader to launch images from external flash

7.4.1 Copy the bootloader project into the application project

To facilitate a seamless usage of the PSOC™ Edge application project and the **PSOC™ Edge Protect Bootloader**, it is essential that the bootloader possesses a comprehensive understanding of the PSOC™ Edge application project's configuration parameters such as the location and sizes of the images. To establish this connection, the bootloader must be imported as a project within the pse_memory_ex3 project.

1. Copy the PSOC_Edge_Protect_Bootloader

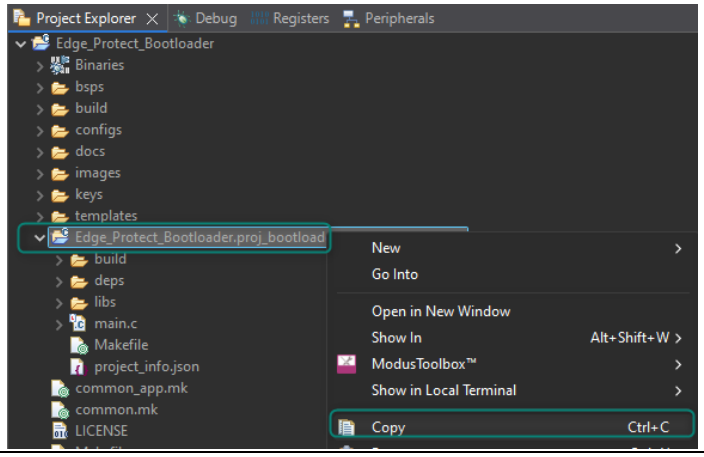


Figure 33 Copying the PSOC™ Edge Protect Bootloader project

2. Paste the **PSOC™ Edge Protect Bootloader** project into the pse_memory_ex3 project

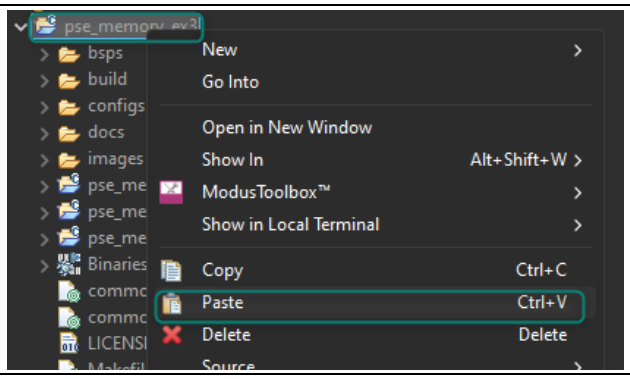


Figure 34 Pasting the PSOC™ Edge Protect Bootloader project

3. While pasting, change the project name to **pse_memory_ex3.proj_bootloader**. Uncheck the **Use Default location** and select the path to the pse_memory_e3 project, and append **\proj_bootloader** to the path as shown in the Figure 35

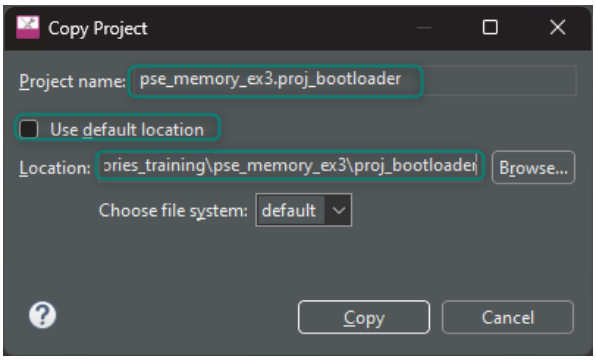


Figure 35 Updating the bootloader project name and location

4. Select the application project, navigate to the quick panel and generate the launch configurations as shown in the Figure 36

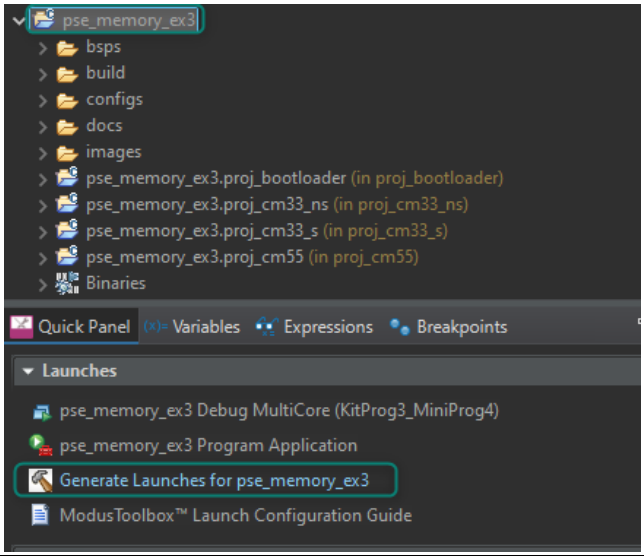


Figure 36 Generate launches

5. Open the *Makefile* at the root of the **pse_memory_ex3** project and add proj_bootloader to the **MTB_PROJECTS** variable

Code listing 29 Updating the application MAKEFILE to add the proj_bootloader

```
MTB_PROJECTS=proj_cm33_s proj_cm33_ns proj_cm55 proj_bootloader
```

7.4.2 Modify the application

1. Open the **Device Configurator** from the Quick Panel
2. Navigate to the **Memory** tab, expand **Serial memory interface block 0, memory 1** and add the secondary image locations for all three application images

Table 9 Secondary regions for bootloaders

Region-Id	Domain	Offset	Size
m33s_secondary	M33S (Domain 0)	0x00880000	0x00240000 (2.25 MB)
m33_secondary	M33 (Domain 1)	0x00AC0000	0x00240000 (2.25 MB)
m55_secondary	M33_M55 (Domain 4)	0x00D00000	0x00300000 (3 MB)

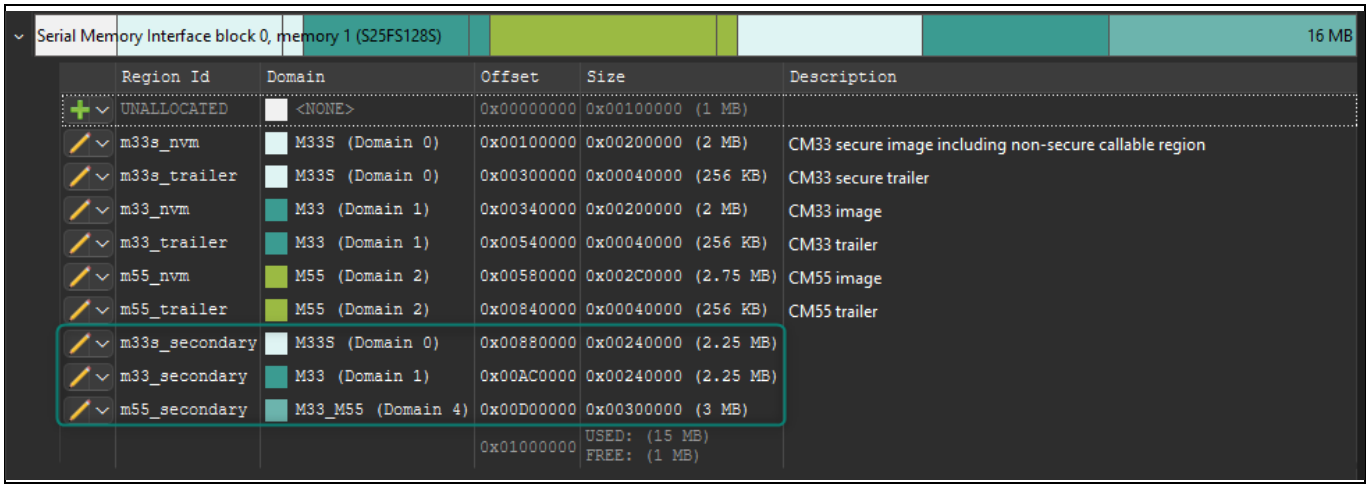


Figure 37 Adding secondary image locations

3. Navigate to the Solutions tab and enable the Edge Protect Bootloader Solution
4. An error message indicates that the **bootloader_nvm** region is missing. Use the quick fix option to add that region

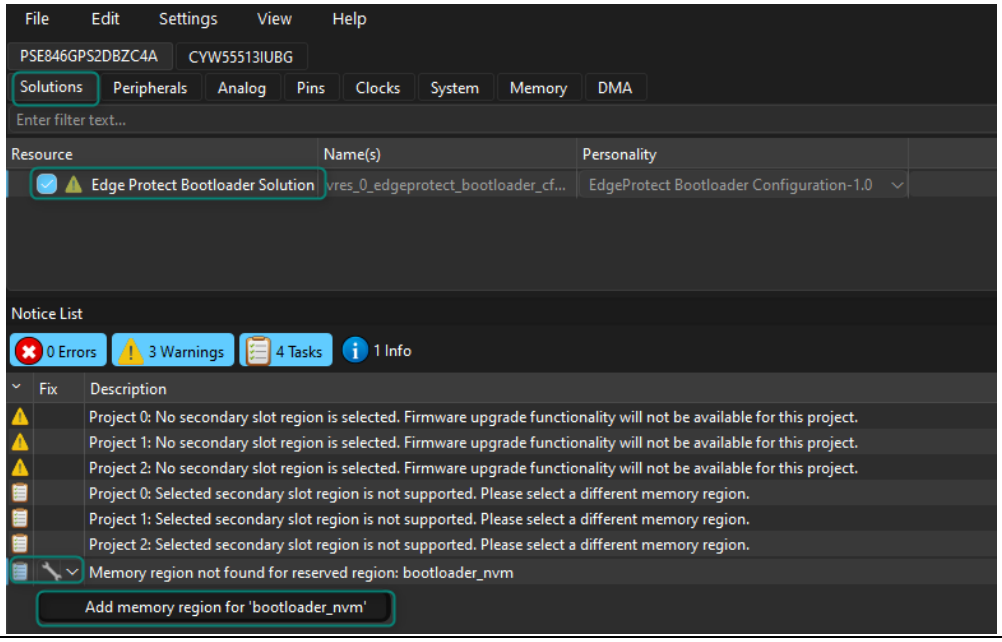


Figure 7 Figure 38Enable Edge Protect Bootloader solution

5. Navigate back to the **Memory** tab, delete the *user_programmable* region, and update the RRAM regions as shown in Table 10

Table 10 RRAM regions using a bootloader

Region-Id	Domain	Offset	Size
bootloader_nvm	M33S (Domain 0)	0x00011000	0x00028000 (160 KB)
user_nvm	M33_M55 (Domain 4)	0x00039000	0x00008000 (32 KB)

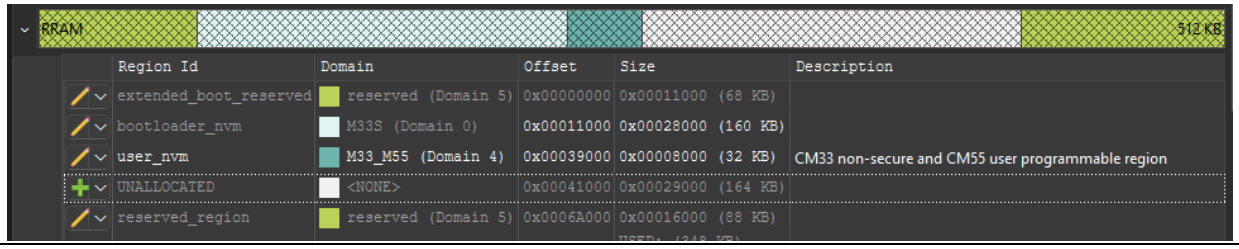


Figure 39 RRAM regions after adding the bootloader

6. Update the Edge Protect Bootloader Solution in the **Solutions** tab

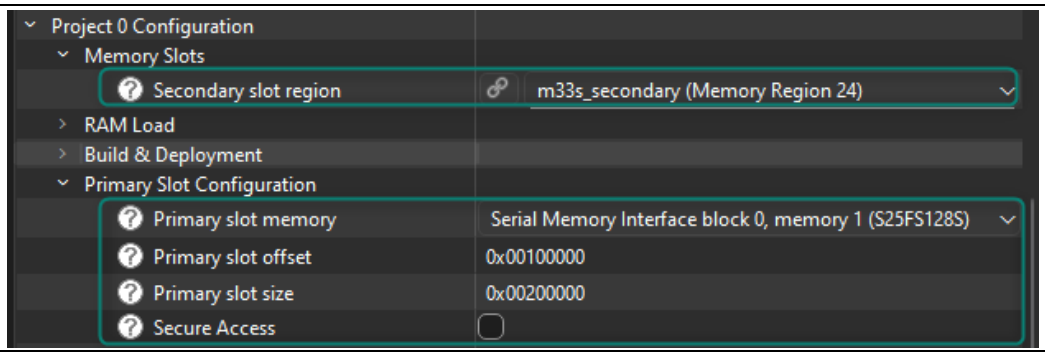


Figure 40 Project 0 configuration for Edge Protect Bootloader solution

Placing and executing code from SRAM

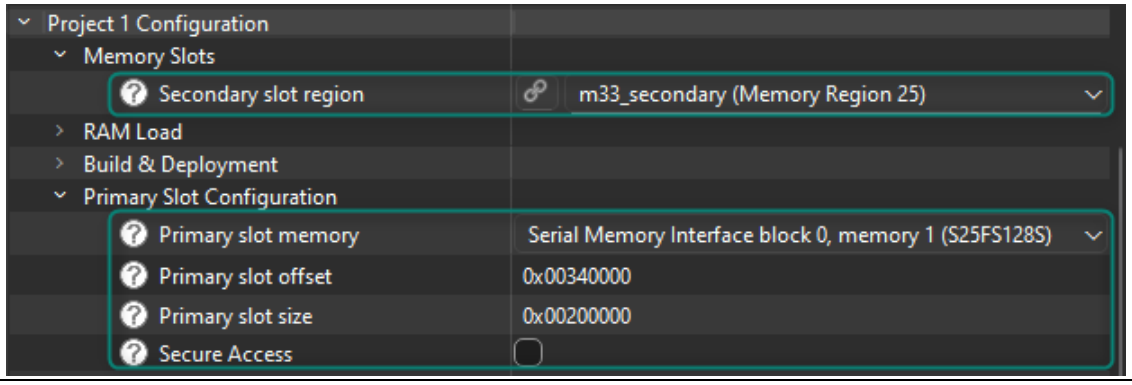


Figure 41 Project 1 configuration for Edge Protect Bootloader solution

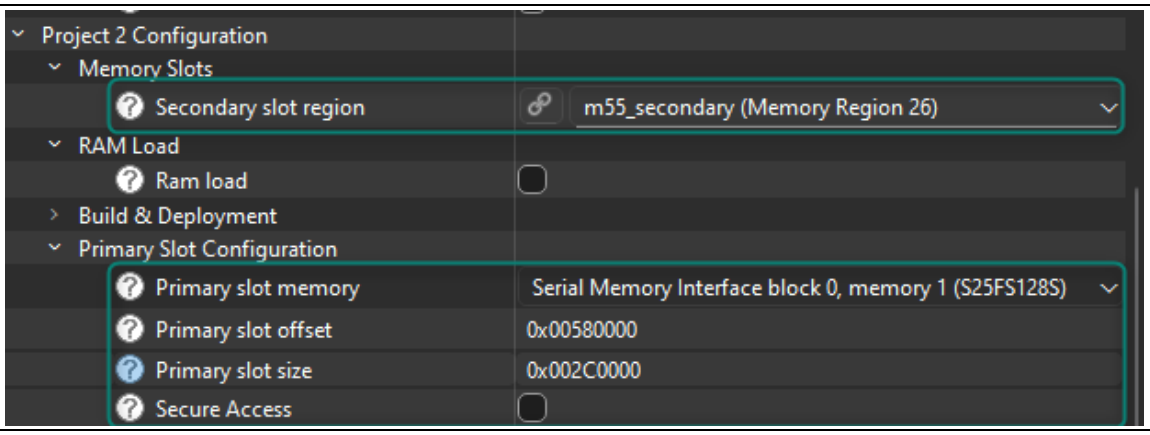


Figure 42 Project 2 configuration for Edge Protect Bootloader Solution

7. Save the design.modus file to regenerate files and close Device Configurator
8. Update the *common.mk* file of the application to use the generated extended boot JSON file

Code listing 30 Change the extended boot configuration file in the common.mk file

```
# Config file for postbuild sign and merge operations.
# NOTE: Check the JSON file for the command parameters
#COMBINE_SIGN_JSON?=configs/boot_with_extended_boot.json
COMBINE_SIGN_JSON?=./bsps/TARGET_APP_KIT_PSE84_EVAL_EPC2/config/GeneratedSource/boot_with_bldr.json
```

9. Build and program the device. The application will execute, and the terminal displays the bootloader output and the **Hello world** message

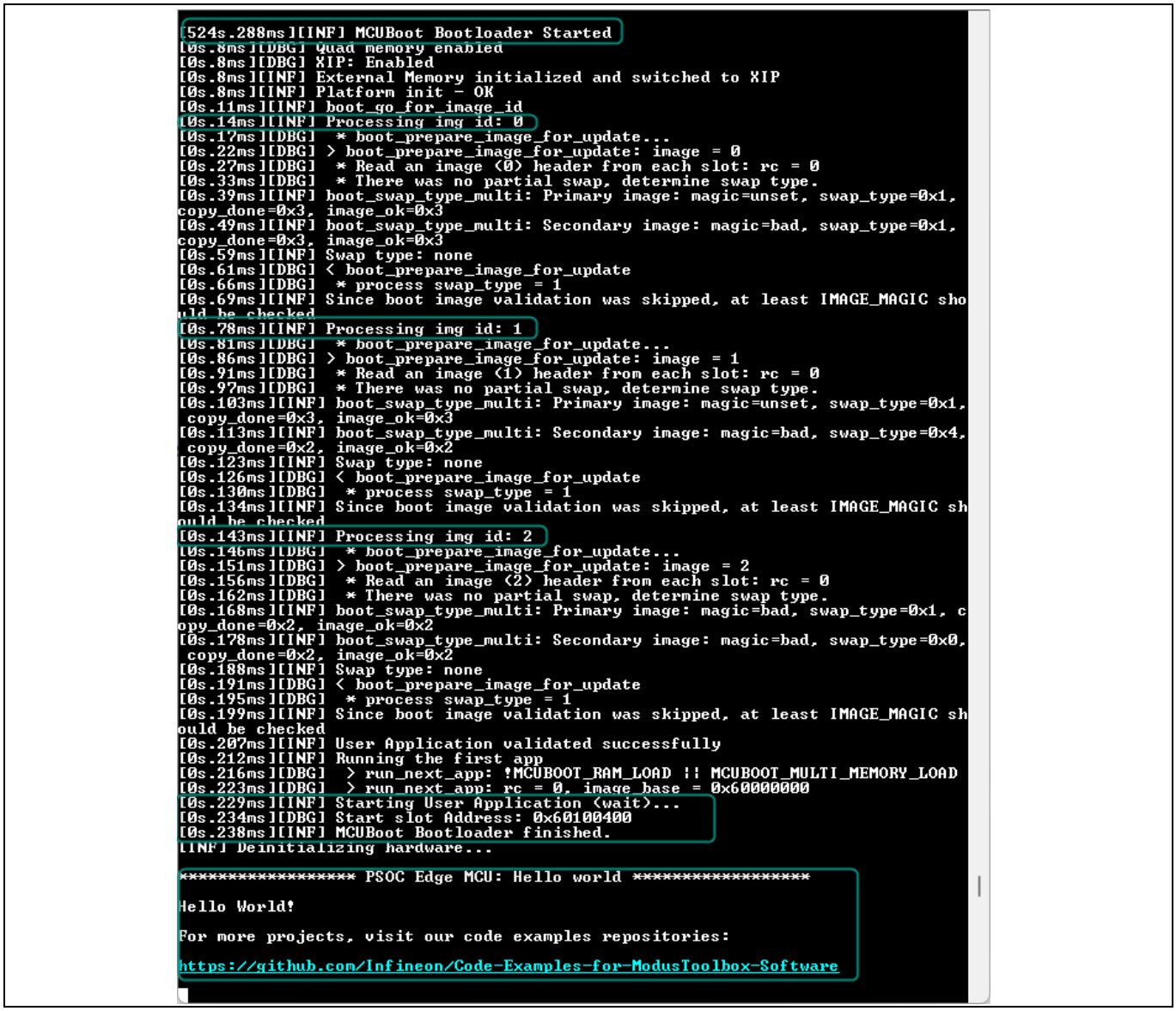


Figure 43 Bootloader log with image processing and application launch

Notice that the user application is launching from 0x60100400. This corresponds to the starting location of the cm33s_nvm region configured in the Device Configurator with an additional 0x400 offset for the MCUBoot header. The MCUBoot header is added by the post-build step, which uses the **COMBINE_SIGN_JSON** to generate the header.

7.5 Using the Edge Protect Bootloader to load application images into SRAM

1. Open the **Device Configurator** from the Quick Panel and navigate to the **Memory** tab
2. Update the **ITCM, DTCM,** and **SRAM** regions to move the *m55_code* and *m55_data* regions from TCM to SRAM memory

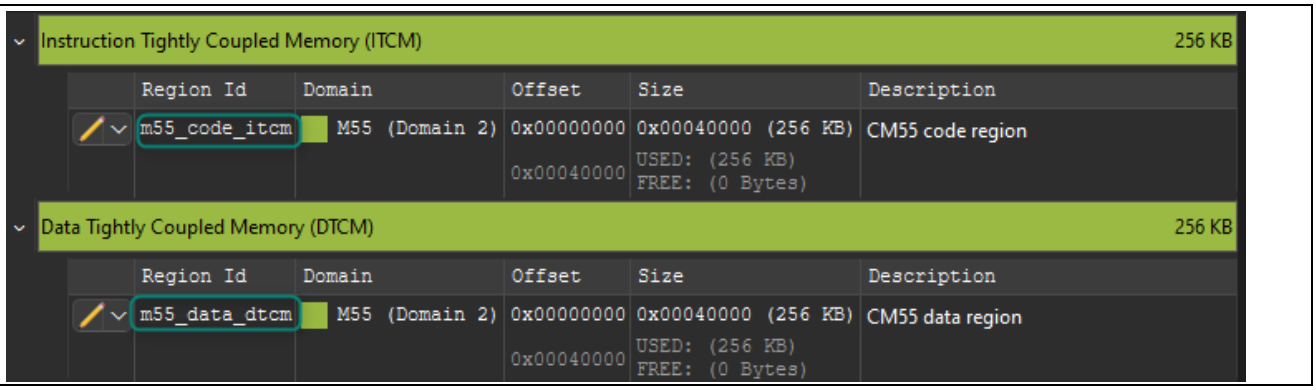


Figure 44 ITCM and DTCM modifications

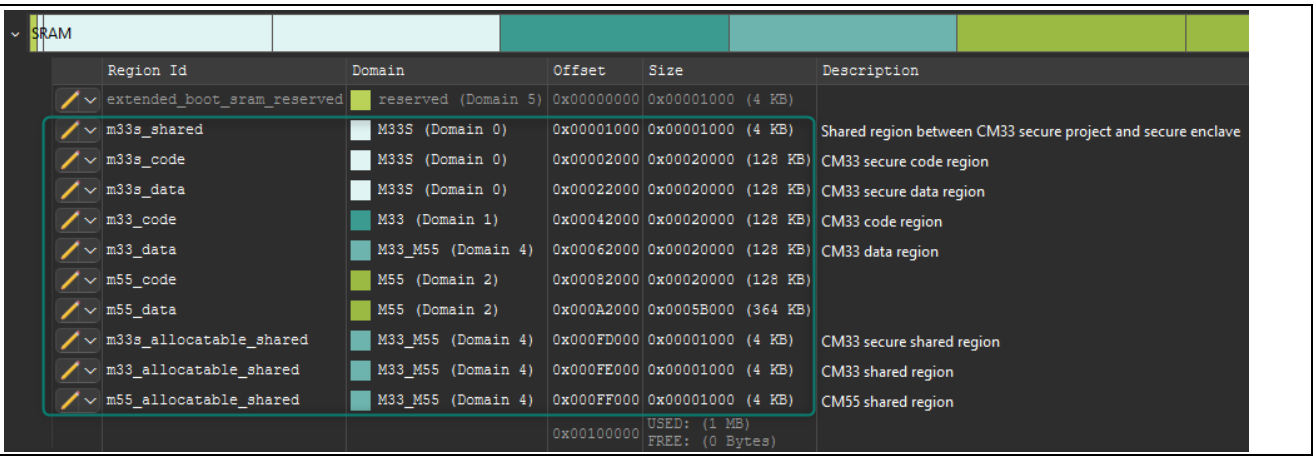


Figure 45 SRAM modifications

3. Navigate to the **Solution** tab and enable **Ram load** for all images, selecting the corresponding memory regions in SRAM

Placing and executing code from SRAM

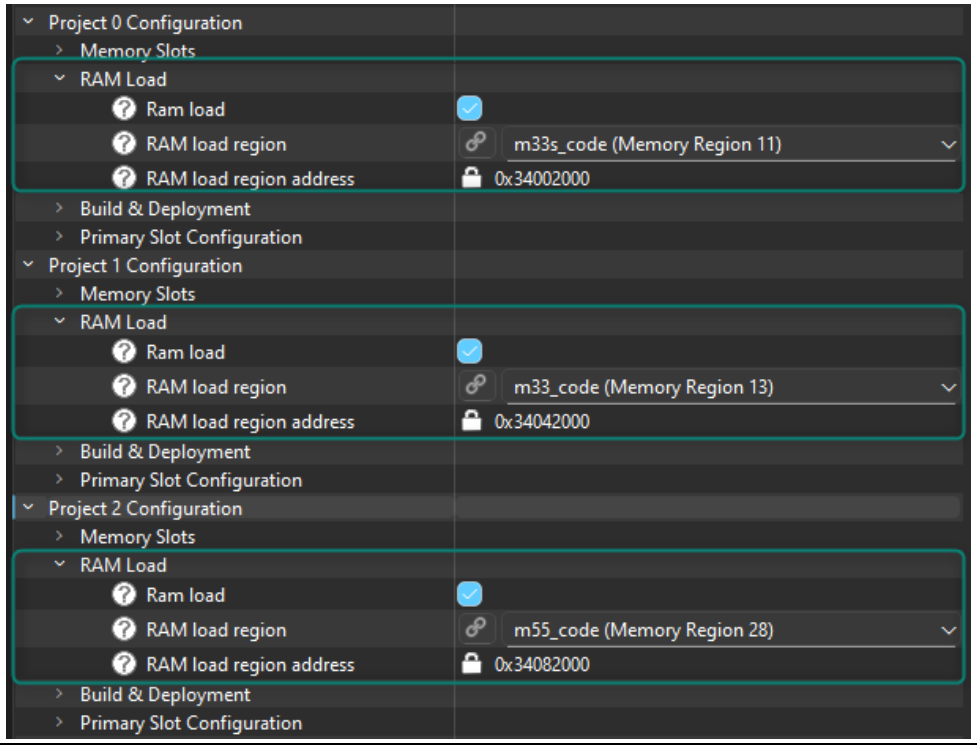


Figure 46 Enabling RAM loading in the Edge Protect Bootloader solution

4. Save the configuration to regenerate files and close **Device Configurator**
5. Build and reprogram the device. The application will execute, and the terminal displays the bootloader output, including RAM loading of each application, followed by the application's hello world message

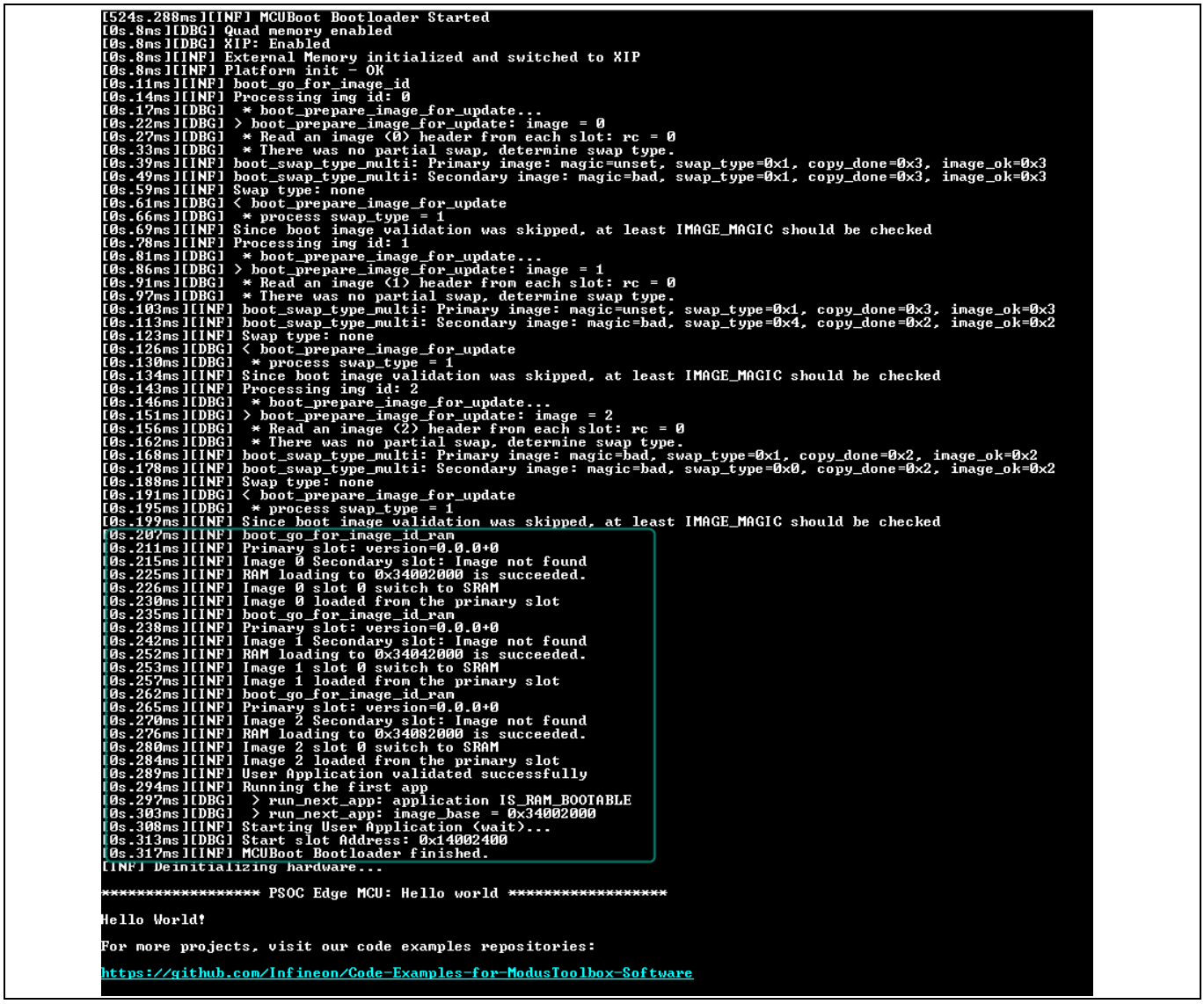


Figure 47 Bootloader log with image processing and application launch

Notice that the user application is now launching from 0x14002400. This corresponds to the starting location of the cm33s_code region configured in the Device Configurator with an additional 0x400 offset for the MCU Boot header. The MCU Boot header is added by the post-build step, which uses the **COMBINE_SIGN_JSON** to generate the header.

7.6 Output

After programming, the application starts automatically. Edge Protect Bootloader runs first, then, based on the configuration, it will either run the CM33_S application in place or copy the application images into SRAM before launching the CM33_S application.



Figure 48 Memory location of the cm33_s application with and without SRAM loading

7.7 Key takeaways

When to use an external flash to load an application to SRAM and run from SRAM?

In embedded systems, consider using external flash to load an application to SRAM and run from SRAM in the following situations:

- **Faster execution required:** Running the application from SRAM can provide faster execution speeds compared to running from external flash memory, which can be beneficial for time-critical applications
- **Low-power consumption during execution:** By loading the application into SRAM, you can reduce the power consumption associated with flash memory accesses during execution, making it suitable for low-power applications

7.8 Conclusion

In this exercise, you successfully used the Edge Protect Bootloader to run an application from external flash, then you updated the device configurator settings so that the bootloader loaded the application images in SRAM before running the code from SRAM. By executing the application from SRAM, you demonstrated the lowest active power consumption and highest execution performance for the CM33 core.

8 Low-power memory considerations

8.1 Objective

This exercise demonstrates how to adjust the PSOC™ Edge low-power mode and how to disable sections of SRAM and SoCMEM to improve the low-power efficiency of the device. The SRAM and System RAM (SoCMEM) regions have multiple configuration options to optimise power consumption while the system is in Deep Sleep. The SRAM and SoCMEM regions are also partitioned into 64 KB and 512 KB sizes, respectively.

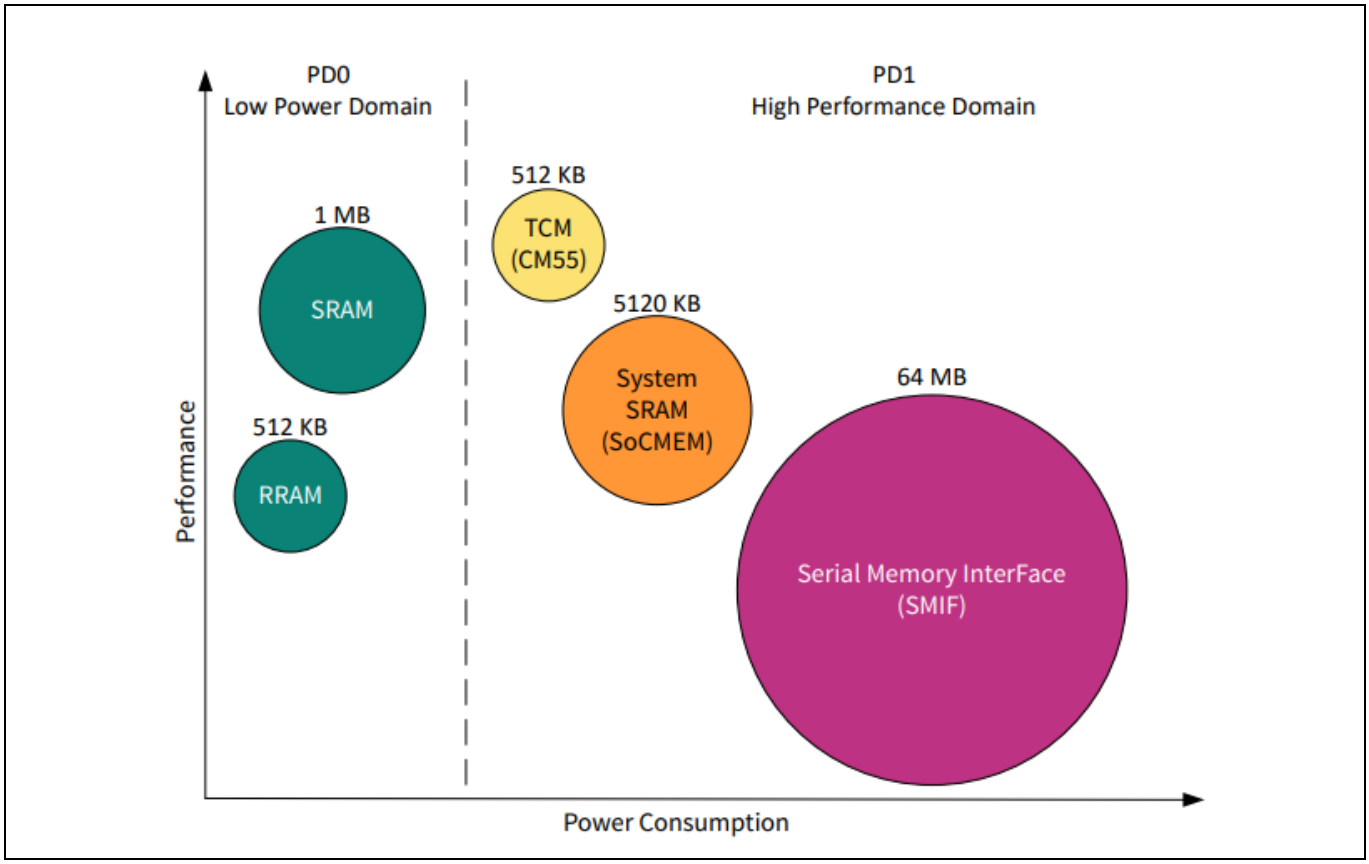


Figure 49 Power consumption vs performance comparison

8.2 Description

By default, all DTCM, SRAM, and SoCMEM regions are enabled. This provides 6.5 MB of total system RAM. When the system is in Deep Sleep mode, only a portion of this memory may be needed. By adjusting what memory regions are retained in system Deep Sleep, the power consumption of the device can be reduced.

This lab demonstrates how to modify the Hello World project to put the device into system Deep Sleep, then measure the baseline power consumption. Next, you will disable SoCMEM before entering system Deep Sleep and compare the power consumption to the baseline. Then you will leave only 512 KB of SoCMEM in retention and measure power consumption. Lastly, you will leave only 64 KB of SRAM and 512 KB of SoCMEM in retention before measuring the power consumption.

8.3 Project creation

Follow steps 1-6 in Appendix A: Creating a PSOC™ Edge application in ModusToolbox™ to create a new application using the Hello World example. Use a descriptive project name, such as **pse_memory_ex4**.

8.4 Baseline memory usage power consumption

1. Open `proj_cm33_ns/main.c` and declare some variables in SRAM and SoCMEM

Code listing 31 Add variables in SRAM and SoCMEM

```
/* App boot address for CM55 project */
#define CM55_APP_BOOT_ADDR          (CYMEM_CM33_0_m55_nvm_START + \
                                     CYBSP_MCUBOOT_HEADER_SIZE)

/*******************************************************************************
* Variables
*******************************************************************************/
CY_SECTION_SHAREDMEM __attribute__((used))  volatile uint32_t shared_sram_variable;
CY_SECTION_SOCMEMSRAMSHARED_BEGIN __attribute__((used))  volatile uint32_t
socmem_variable;
```

2. Update the for-loop to print the variable value and location before entering system Deep Sleep

Note: By default, the hello world project will put the CM55 into Deep Sleep mode, so the only edit needed to put the device into system Deep Sleep is to put the CM33 into Deep Sleep.

Code listing 32 Print variable values and locations before entering Deep Sleep

```
    shared_sram_variable = 0xDEADBEEF;
    socmem_variable = 0xBEADCAFE;

    for(;;)
    {
        printf("Shared SRAM Variable: 0x%08X; Location: 0x%08X \n",
               shared_sram_variable,
               (unsigned int)&shared_sram_variable);
        printf("SoCMEM Variable: 0x%08X; Location: 0x%08X \n\n",
               socmem_variable,
               (unsigned int)&socmem_variable);

        while(!(Cy_SCB_UART_IsTxComplete(CYBSP_DEBUG_UART_HW))) {};

        Cy_GPIO_Inv(CYBSP_USER_LED1_PORT, CYBSP_USER_LED1_PIN);

        Cy_SysLib_Delay(500);

        Cy_SysPm_CpuEnterDeepSleep(CY_SYSPM_WAIT_FOR_INTERRUPT);
    }
```

Low-power memory considerations

3. Enable a GPIO as an interrupt to wake the device from system Deep Sleep
   a. Add the GPIO ISR

Code listing 33 GPIO interrupt service routine

```
/*******************************************************************************
* Function Name: gpio_isr_handler
********************************************************************************
* Summary:
*  Interrupt service routine for CYBSP_USER_BTN_PORT and CYBSP_USER_BTN2_PORT.
*  This function checks which user button was pressed and clears the
*  interrupt.
*
* Parameter:
*  void
*
* Return:
*  void
*
*******************************************************************************/
void gpio_isr_handler(void)
{
    /* Check if USER_BTN1 was pressed */
    if(1UL == Cy_GPIO_GetInterruptStatus(CYBSP_USER_BTN_PORT, CYBSP_USER_BTN_PIN))
    {
        /* Clear the USER_BTN1 interrupt */
        Cy_GPIO_ClearInterrupt(CYBSP_USER_BTN_PORT, CYBSP_USER_BTN_PIN);
    }

    /* Check if USER_BTN2 was pressed */
    if(1UL == Cy_GPIO_GetInterruptStatus(CYBSP_USER_BTN2_PORT,
CYBSP_USER_BTN2_PIN))
    {
        /* Clear the USER_BTN2 interrupt */
        Cy_GPIO_ClearInterrupt(CYBSP_USER_BTN2_PORT, CYBSP_USER_BTN2_PIN);
    }
}
```

b. Then, add the user code to enable the interrupt

Code listing 34 Enable GPIO interrupt

```
int main(void)
{
    cy_rslt_t result = CY_RSLT_SUCCESS;
    cy_en_sysint_status_t int_status;

    cy_stc_sysint_t gpio_int_config =
    {
        .intrSrc = CYBSP_USER_BTN1_IRQ,
        .intrPriority = 7U,
    };

    /* Initialize the device and board peripherals. */
    result = cybsp_init();

    /* Board initialization failed. Stop program execution. */
    if (CY_RSLT_SUCCESS != result)
    {
        handle_app_error();
    }

     `
   int_status = Cy_SysInt_Init(&gpio_int_config, gpio_isr_handler);
    if(CY_SYSINT_SUCCESS != int_status)
    {
        handle_app_error();
    }

    NVIC_EnableIRQ(gpio_int_config.intrSrc);

    /* Enable global interrupts */
    __enable_irq();
```

4. Program the device. Observe that the data in the variables are retained while the device is in the system Deep Sleep after a press button that wakes the device from system Deep Sleep and prints the values again

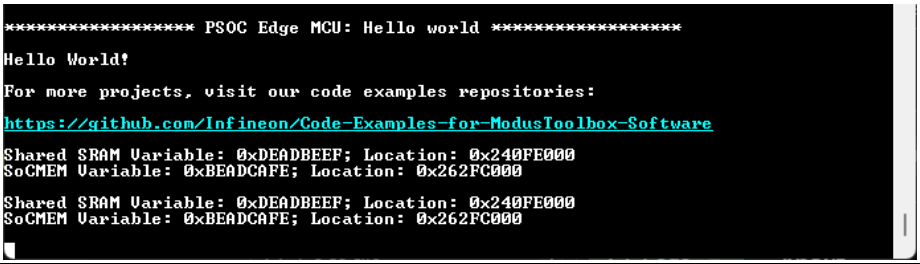


Figure 50 Terminal output

5. Measure power consumption and record it in Table 13

The SOM board included with the PSOC™ Edge EVK is pre-configured in battery-powered mode, meaning that the device has a 3.3 V component (VBAT) and a 1.8 V component (VDDD, VDDQ, VDDIO, VCCD, and so on).

To measure the two power rails, use J25 and J26. See Appendix B: KIT_PSE84_EVAL details.

- J25 measures IDDD/IDDIO (1.8 V)
- J26 measures IBAT (3.3 V)

Note: The current measurement on J25 includes IDDIO and extra current from other EVK components. Some rework can be implemented as explained in the mtb-example-psoc-edge-power-measurements code example; J25 provides an easier measurement for training purposes.

Note: The power consumption of this example is not fully optimized, but it's intended to show the difference when enabling memory retention. ModusToolbox™ includes more optimized examples, such as mtb-example-psoc-edge-power-measurements.

8.5 Disabling SoCMEM

1. In `proj_cm33_ns/main.c`, add the following code to disable System SRAM (SoCMEM)

Code listing 35 Disable System SRAM

```
/* Enable CM55. */
/* CM55_APP_BOOT_ADDR must be updated if CM55 memory layout is changed.*/
Cy_SysEnableCM55(MXCM55, CM55_APP_BOOT_ADDR, CM55_BOOT_WAIT_TIME_USEC);

/* Disable System SRAM (SoCMEM) */
Cy_SysEnableSOCMEM(false);
```

2. Program the device and press the user button. Observe that the SRAM variable value is retained, but that SoCMEM variable is not

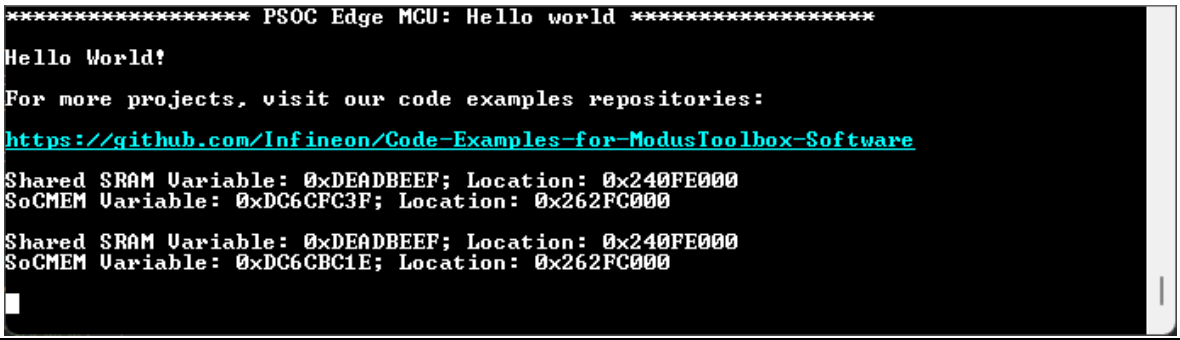


Figure 51 Terminal output with SoCMEM disabled

3. Measure power consumption and record in Table 13

Note: When SoCMEM is disabled, the variable in SoCMEM is not retained even while the CM33 is active.

8.6 SoCMEM retention

The SoCMEM partitions can be enabled or disabled in active power mode. They can also be set up for retention or no retention while the device is in system Deep Sleep. Let's see how leaving only one partition in retention impacts the power consumption in the system Deep Sleep.

Based on the location of the SoCMEM variable (0x262FC000), we can observe that it is in the fifth SoCMEM partition. See the Table 11 for all SoCMEM partition addresses.

Table 11 SoCMEM partitions

Partition	Start	End
CY_SYSPM_SOCMEM_SRAM_PARTITION0	0x26000000	0x2607FFFF
CY_SYSPM_SOCMEM_SRAM_PARTITION1	0x26080000	0x260FFFFF
CY_SYSPM_SOCMEM_SRAM_PARTITION2	0x26100000	0x2617FFFF
CY_SYSPM_SOCMEM_SRAM_PARTITION3	0x26180000	0x261FFFFF
CY_SYSPM_SOCMEM_SRAM_PARTITION4	0x26200000	0x2627FFFF
CY_SYSPM_SOCMEM_SRAM_PARTITION5	0x26280000	0x262FFFFF
CY_SYSPM_SOCMEM_SRAM_PARTITION6	0x26300000	0x2637FFFF
CY_SYSPM_SOCMEM_SRAM_PARTITION7	0x26380000	0x263FFFFF
CY_SYSPM_SOCMEM_SRAM_PARTITION8	0x26400000	0x2647FFFF
CY_SYSPM_SOCMEM_SRAM_PARTITION9	0x26480000	0x264FFFFF

Low-power memory considerations

1. Add a new variable and place it in the user-defined “gfx_mem” region in SoCMEM

Code listing 36 Add a variable in gfx_mem region of SoCMEM

```
/*******************************************************************************
* Variables
*******************************************************************************/
CY_SECTION_SHAREDMEM __attribute__((used))  volatile uint32_t shared_sram_variable;
CY_SECTION_SOCMEMSRAMSHARED_BEGIN __attribute__((used))  volatile uint32_t
socmem_variable;
CY_SECTION(".cy_gpu_buf") __attribute__((used))  volatile uint32_t
socmem_gfx_variable;
```

2. Comment the *Cy_SysEnableSOCMEM* API to re-enable SoCMEM and print the new variable and its location

Code listing 37 Enable SoCMEM and print new variable

```
/* Disable System SRAM (SoCMEM) */
//Cy_SysEnableSOCMEM(false);

shared_sram_variable = 0xDEADBEEF;
socmem_variable = 0xBEADCAFE;
socmem_gfx_variable = 0xCAFEBEEF;

for(;;)
{
    printf("Shared SRAM Variable: 0x%08X; Location: 0x%08X \n",
           shared_sram_variable,
           (unsigned int)&shared_sram_variable);
    printf("SoCMEM Variable: 0x%08X; Location: 0x%08X \n",
           socmem_variable,
           (unsigned int)&socmem_variable);
    printf("GFX SoCMEM Variable: 0x%08X; Location: 0x%08X \n\n",
            socmem_gfx_variable,
           (unsigned int) &socmem_gfx_variable);
```

3. Add user code to disable all partitions except partition 5

Code listing 38 Disable SoCMEM partitions, except for partition #9

```
/* Enable CM55. */
/* CM55_APP_BOOT_ADDR must be updated if CM55 memory layout is changed.*/
Cy_SysEnableCM55(MXCM55, CM55_APP_BOOT_ADDR, CM55_BOOT_WAIT_TIME_USEC);

#define SOCMEM_PARTITIONS_TO_DISABLE    (9U)
uint8_t socmem_partitions_to_disable[SOCMEM_PARTITIONS_TO_DISABLE] =
```

```
    {CY_SYSPM_SOCMEM_SRAM_PARTITION0,
    CY_SYSPM_SOCMEM_SRAM_PARTITION1,
    CY_SYSPM_SOCMEM_SRAM_PARTITION2,
    CY_SYSPM_SOCMEM_SRAM_PARTITION3,
    CY_SYSPM_SOCMEM_SRAM_PARTITION4,
    CY_SYSPM_SOCMEM_SRAM_PARTITION6,
    CY_SYSPM_SOCMEM_SRAM_PARTITION7,
    CY_SYSPM_SOCMEM_SRAM_PARTITION8,
    CY_SYSPM_SOCMEM_SRAM_PARTITION9};


for (uint8_t i = 0; i < SOCMEM_PARTITIONS_TO_DISABLE; i++)
{
    Cy_SysPm_SetSOCMemPartDsPwrMode(socmem_paritions_to_disable[i],
CY_SYSPM_SOCMEM_SRAM_DS_MODE_RET_OFF);
}
```

4. Program the device. Observe that the SRAM and SoCMEM variable values are retained, but the GFX SoCMEM variable is not retained

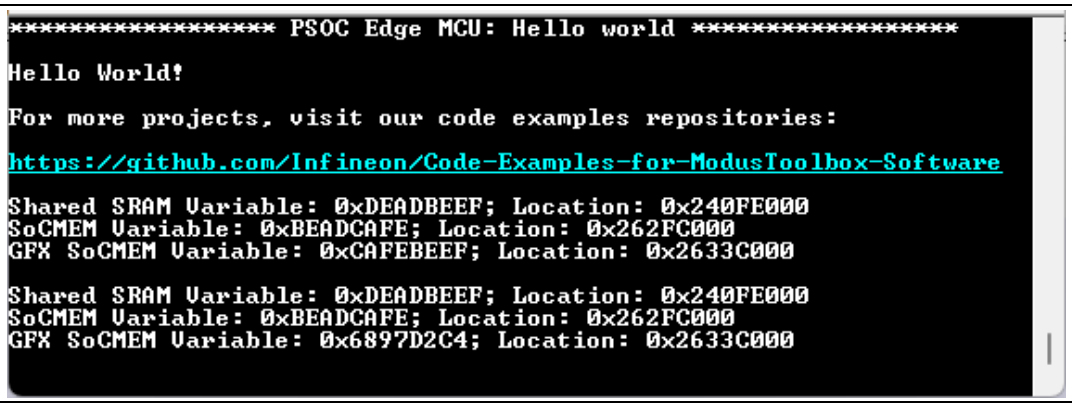


Figure 8 Figure 52 Debug output with all but one SoCMEM partition disabled

5. Measure power consumption and record in Table 13

Note: The variable in the "gfx_mem" region of SoCMEM is available while the device is not in system Deep Sleep, but that the value is not retained through system Deep Sleep. This is useful for RAM variables that do not need to be maintained when the system is in the Deep Sleep. Reducing the number of SoCMEM partitions in retention while in the system Deep Sleep will reduce the power consumed by the device.

8.7 SRAM retention

The SRAM partitions on PSOC™ Edge can either be enabled or disabled. The PSOC™ Edge EVK board support package (BSP) configures the SRAM as shown in the Figure 53.

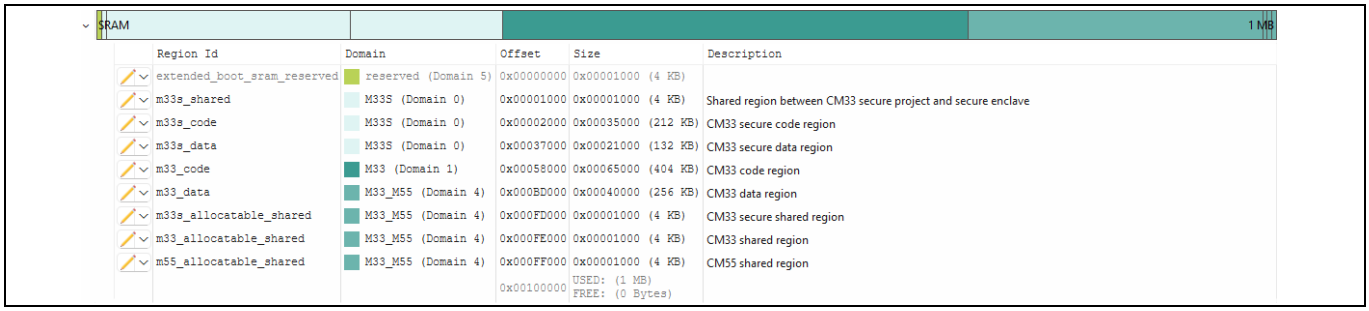


Figure 53 Default SRAM memory allocations

Note: The m33_data region is not aligned with an SRAM bank partition; instead, it spans SRAM 1 partitions 3 to 7.

Table 12 SRAM partitions

SRAM Bank	Partition	Start	End
SRAM 0	0	0x24000000	0x2400FFFF
SRAM 0	1	0x24010000	0x2401FFFF
SRAM 0	2	0x24020000	0x2402FFFF
SRAM 0	3	0x24030000	0x2403FFFF
SRAM 0	4	0x24040000	0x2404FFFF
SRAM 0	5	0x24050000	0x2405FFFF
SRAM 0	6	0x24060000	0x2406FFFF
SRAM 0	7	0x24070000	0x2407FFFF
SRAM 1	0	0x24080000	0x2408FFFF
SRAM 1	1	0x24090000	0x2409FFFF
SRAM 1	2	0x240A0000	0x240AFFFF
SRAM 1	3	0x240B0000	0x240BFFFF
SRAM 1	4	0x240C0000	0x240CFFFF
SRAM 1	5	0x240D0000	0x240DFFFF
SRAM 1	6	0x240E0000	0x240EFFFF
SRAM 1	7	0x240F0000	0x240FFFFF

In order for the device to properly enter and exit Deep Sleep mode, the CM33 stack and other corresponding variable must be available to the CM33. These variables are by default stored in the cm33_data region of SRAM.

Let's modify the cm33_data SRAM region so that it fits into one SRAM partition, then update the source code to print out SRAM variables in the cm33_data region and the m33_allocatable_shared region. Then we will set only the SRAM partition for the cm33_data in retention mode.

Low-power memory considerations

1. Open the **Device Configurator** from the Quick Panel, then go to the **Memory** tab and update the **m33_data** region in SRAM so that it aligns with SRAM 1 partition 4 with an offset of 0x000C0000

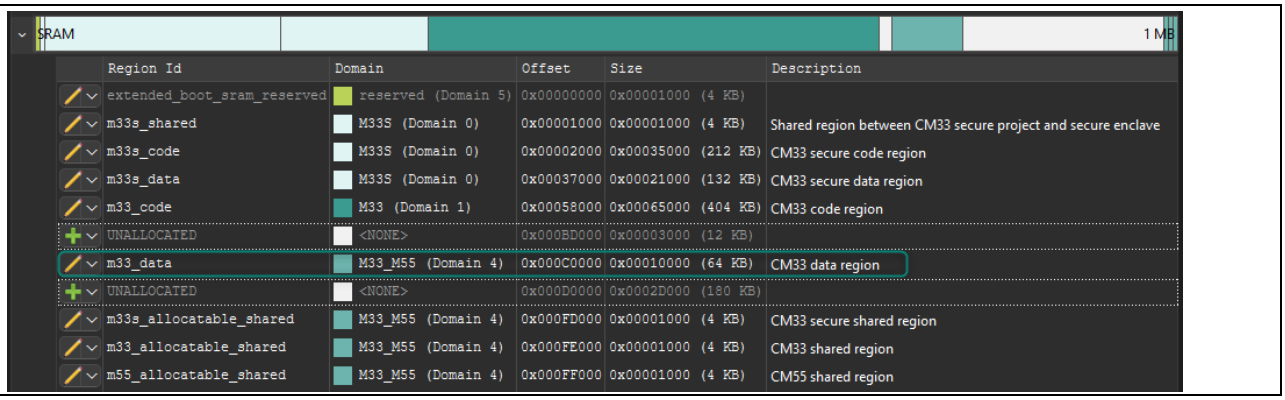


Figure 54 Updated SRAM memory allocation for cm33_data region

2. Save the configuration to regenerate files and close Device Configurator
3. In `proj_cm33_ns/main.c`, add a new SRAM variable

Code listing 39 Add SRAM variable

```
/*******************************************************************************
* Variables
*******************************************************************************/
volatile uint32_t sram_variable;
CY_SECTION_SHAREDMEM __attribute__((used))  volatile uint32_t shared_sram_variable;
CY_SECTION_SOCMEMSRAMSHARED_BEGIN __attribute__((used))  volatile uint32_t
socmem_variable;
CY_SECTION(".cy_gpu_buf") __attribute__((used))  volatile uint32_t
socmem_gfx_variable;
```

4. Print the new variables and their location

Code listing 40 Print variables and locations

```
sram_variable = 0xFEEDBEEF;
shared_sram_variable = 0xDEADBEEF;
socmem_variable = 0xBEADCAFE;
socmem_gfx_variable = 0xCAFEBEEF;

printf("Values before turning off SRAM \n");
printf("SRAM Variable: 0x%08X; Location: 0x%08X \n",
       sram_variable,
       (unsigned int) &sram_variable);
printf("Shared SRAM Variable: 0x%08X; Location: 0x%08X \n",
       shared_sram_variable,
       (unsigned int)&shared_sram_variable);
printf("SoCMEM Variable: 0x%08X; Location: 0x%08X \n",
```

```
        socmem_variable,
        (unsigned int)&socmem_variable);
printf("GFX SoCMEM Variable: 0x%08X; Location: 0x%08X \n\n",
        socmem_gfx_variable,
        (unsigned int) &socmem_gfx_variable);
```

5. Add a delay to ensure that the CM55 is in Deep Sleep Mode before disabling SRAM partitions

Code listing 41 Wait for the CM55 to go to Deep Sleep

```
/* Enable CM55. */
/* CM55_APP_BOOT_ADDR must be updated if CM55 memory layout is changed.*/
Cy_SysEnableCM55(MXCM55, CM55_APP_BOOT_ADDR, CM55_BOOT_WAIT_TIME_USEC);

uint32_t cpu_state = CY_SYS_CORE_STATUS_ACTIVE;

while(CY_SYS_CORE_STATUS_DEEPSLEEP != cpu_state)
{
    cpu_state = Cy_SysGetCM55Status(MXCM55);
}
```

6. Set all SRAM partitions except SRAM 1 partition 4 into "CY_SYSPM_SRAM_PWR_MODE_OFF"

Code listing 42 Set all SRAM partitions to PWR MODE OFF except SRAM 1 partition 4

```
for (uint8_t i = 0; i < 8; i++)
{
    Cy_SysPm_SetSRAMMacroPwrMode(CY_SYSPM_SRAM0_MEMORY, i,
                                 CY_SYSPM_SRAM_PWR_MODE_OFF);
}
for (uint8_t i = 0; i < 4; i++)
{
    Cy_SysPm_SetSRAMMacroPwrMode(CY_SYSPM_SRAM1_MEMORY, i,
                                 CY_SYSPM_SRAM_PWR_MODE_OFF);
}
for (uint8_t i = 5; i < 8; i++)
{
    Cy_SysPm_SetSRAMMacroPwrMode(CY_SYSPM_SRAM1_MEMORY, i,
                                 CY_SYSPM_SRAM_PWR_MODE_OFF);
}
for(;;)
{
```

7. Print the variable values and locations after exiting Deep Sleep mode

Code listing 43 Print variable value and locations after exiting Deep Sleep

```
for(;;)
{
    while(!(Cy_SCB_UART_IsTxComplete(CYBSP_DEBUG_UART_HW))) {};

    Cy_GPIO_Inv(CYBSP_USER_LED1_PORT, CYBSP_USER_LED1_PIN);
    Cy_SysLib_Delay(500);
    Cy_SysPm_CpuEnterDeepSleep(CY_SYSPM_WAIT_FOR_INTERRUPT);

    printf("Values after waking-up from Deep Sleep\n");
    printf("SRAM Variable: 0x%08X; Location: 0x%08X \n",
           sram_variable, (unsigned int) &sram_variable);
    printf("Shared SRAM Variable: 0x%08X; Location: 0x%08X \n",
           shared_sram_variable, (unsigned int)&shared_sram_variable);
    printf("SoCMEM Variable: 0x%08X; Location: 0x%08X \n",
           socmem_variable, (unsigned int)&socmem_variable);
    printf("GFX SoCMEM Variable: 0x%08X; Location: 0x%08X \n\n",
           socmem_gfx_variable, (unsigned int) &socmem_gfx_variable);
}
```

8. Program the device. Observe the variable outputs and locations, then press SW2 and see that only the SRAM variable in cm33_data and the SoCMEM variable are retained

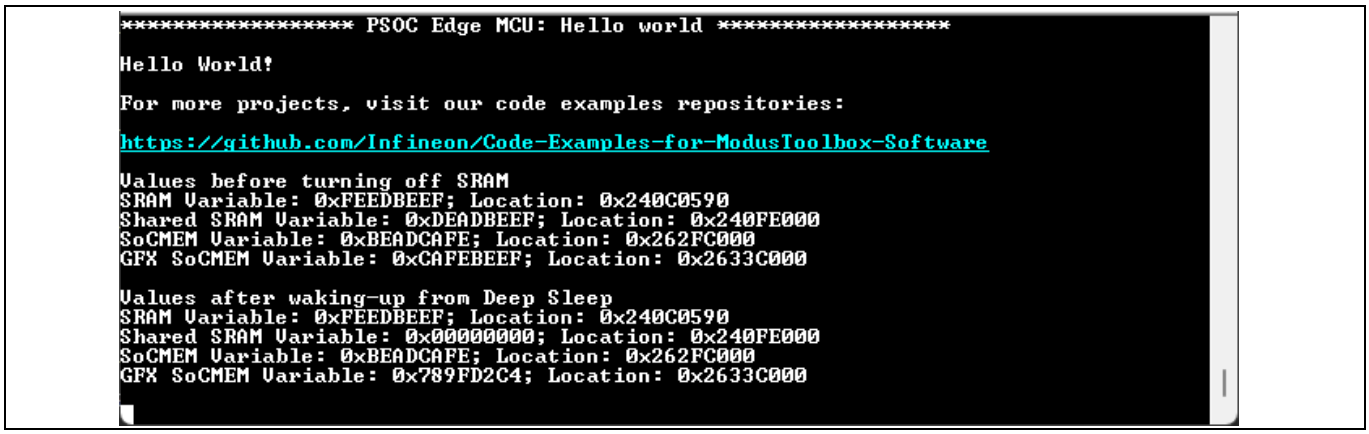


Figure 55 Terminal output with all but one SoCMEM and one SRAM partition disabled

9. Measure power consumption and record in Table 13

8.8 Output

Use the Table 13 to note down your power consumption measurements from each step in this lab.

The SOM board included with the PSOC™ Edge EVK is pre-configured in battery-powered mode, meaning that the device has a 3.3V component (VBAT), and a 1.8V component (VDDD, VDDQ, VDDIO, VCCD, and so on).

To measure the two power rails, use J25 and J26. See Appendix B: KIT_PSE84_EVAL details.

– J25 measures IDDD/IDDIO (1.8 V)

– J26 measures IBAT (3.3 V)

Note: The current measurement on J25 includes IDDIO and extra current from other EVK components. Some rework can be implemented as explained in the mtb-example-psoc-edge-power-measurements code example; J25 provides an easier measurement for training purposes.

Table 13 Power consumption measurements

Lab step	IDDD (1.8 V)		IBAT (3.3 V)		Total power	
	Expected	Measured	Expected	Measured	Expected	Measured
Baseline	~213 µA	–	~86 µA	–	~667 µW	–
SoCMEM disabled	~213 µA	–	~59 µA	–	~578 µW	–
512 KB of SoCMEM in retention	~213 µA	–	~62 µA	–	~588 µW	–
512 KB of SoCMEM and 64 KB of SRAM in retention	~213 µA	–	~57 µA	–	~571 µW	–

Note: The Hello World project is not optimized for power consumption, but the relative changes in power consumption are relevant. Lowering SRAM availability will reduce power consumption.

8.9 Takeaways

When do you adjust the SoCMEM retained?

- SoCMEM is a 5 MB block of SRAM in the high performance PD1 domain. It is useful when there is a need to buffer large amounts of volatile data such as graphics buffers and machine learning arenas. However, SoCMEM has the highest power consumption of internal SRAM blocks available. When trying to achieve low-power consumption, it is best to disable SoCMEM retention where possible
- In certain scenarios where the device may need to wake quickly from Deep Sleep, perform a high-performance processing task, then go back to Deep Sleep, it may be beneficial to leave some SoCMEM in retention while the device is in Deep Sleep so that whatever data is buffered does not need to be regenerated. This methodology can be used to keep the average power consumption low by keeping the time that the processor is on short

When do you adjust what SRAM partitions are enabled?

- While SRAM consumes less power than SoCMEM, it does still draw current to retain the data while in system Deep Sleep. If the application is in a low-power state, it is beneficial to limit the amount of SRAM enabled, but the CM33 stack is stored in SRAM, so at least 64 KB must be enabled for the CPU to operate

8.10 Conclusion

In this exercise you placed variables in different SRAM and SoCMEM locations, then adjusted the SoCMEM Deep Sleep retention and reduced the SRAM partitions enabled to reduce the power consumed by the device.

9 Appendix A: Creating a PSOC™ Edge application in ModusToolbox™

The following steps show how to create a new project for PSOC™ Edge in ModusToolbox™.

1. Open the **ModusToolbox™ Dashboard 3.6** and launch the **Eclipse IDE for ModusToolbox™**

   Note: If you have not installed ModusToolbox™ or Eclipse IDE, see Required development tools section.

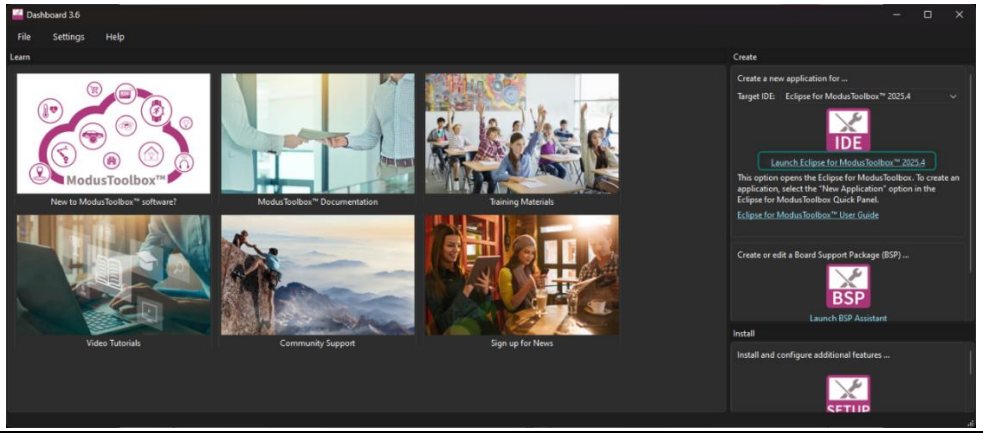


Figure 56 Open Eclipse IDE for ModusToolbox™ from Dashboard

2. Choose a **workspace directory** for your project. A folder will be created if the workspace doesn't exist. After selecting a suitable directory, click the **Launch** button. It is recommended to keep the workspace path short. Windows has a 260-character length limit and a long workspace path may cause build issues when creating some applications

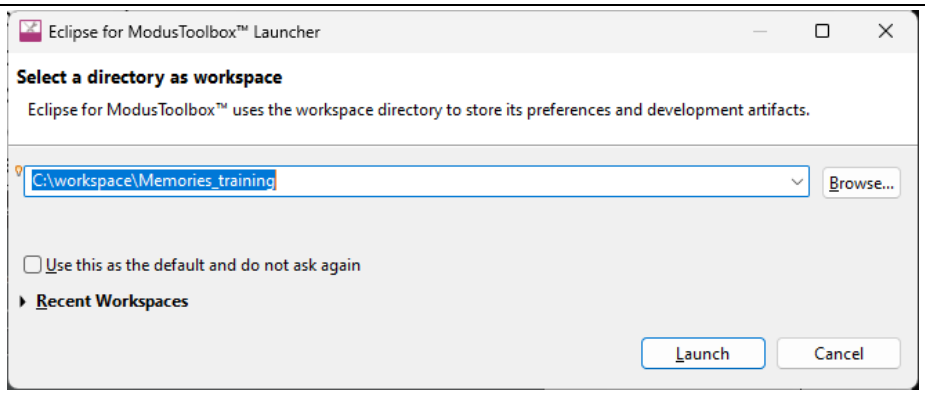


Figure 57 Choose workspace directory

3. Select **New Application** from the Quick Panel. Alternatively, go to **File> New> ModusToolbox™** application. This will launch the **Project Creator Tool**

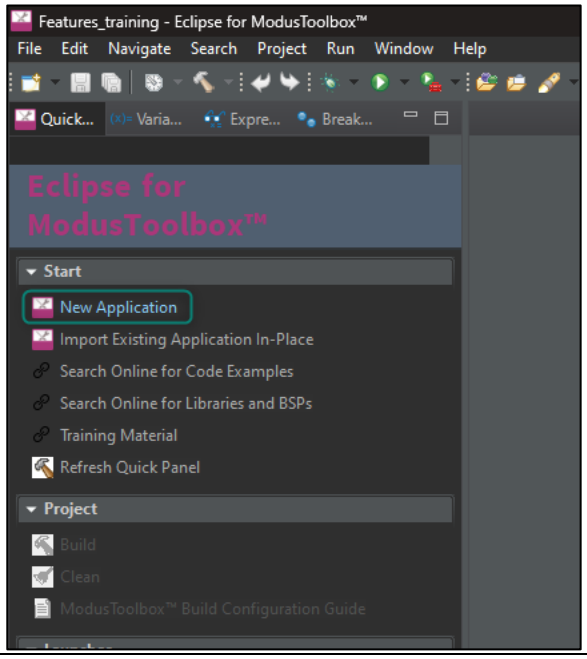


Figure 58 Create New application

4. Once **Projector Creator** launches, you'll be prompted to select a board support package (BSP).
   Select the **KIT_PSE84_EVAL_EPC2 BSP** under the PSOC™ Edge BSPs and click **Next.**
   BSPs are aligned with our development/evaluation kits; they provide files for basic device functionality. A BSP typically has a **design.modus** file that configures clocks and other board-specific capabilities. That file is used by the ModusToolbox™ configurators. A BSP also includes the required device support code for the device on the board. You can modify the configuration to suit your application. Observe that ModusToolbox™ also includes BSPs for PSOC™ Edge EPC4 devices (KIT_PSE84_EVAL_EPC4), and for the PSOC™ Edge AI Kit (KIT_PSE84_AI); however, this training is intended to be used with the KIT_PSE84_EVAL, which includes EPC2 devices by default

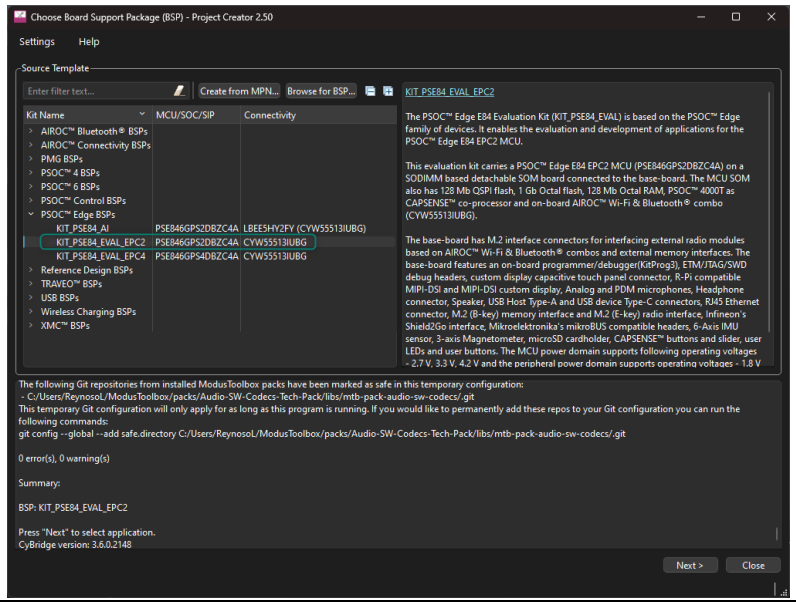


Figure 59 Selecting BSP in Project Creator

Appendix A: Creating a PSOC™ Edge application in ModusToolbox™

5. A new window opens to select an application. The Project Creator includes categories and a search field to make it easier to find and select different examples.

   Click the **checkbox** near the application, optionally rename the project, and click **Create**.

   Figure 60 shows the creation of the **PSOC™ Edge Hello World** application under the **Getting Started section**.

   Windows has a 260-character path length limit. A long workspace path and/or a long project name may cause build issues when creating some applications

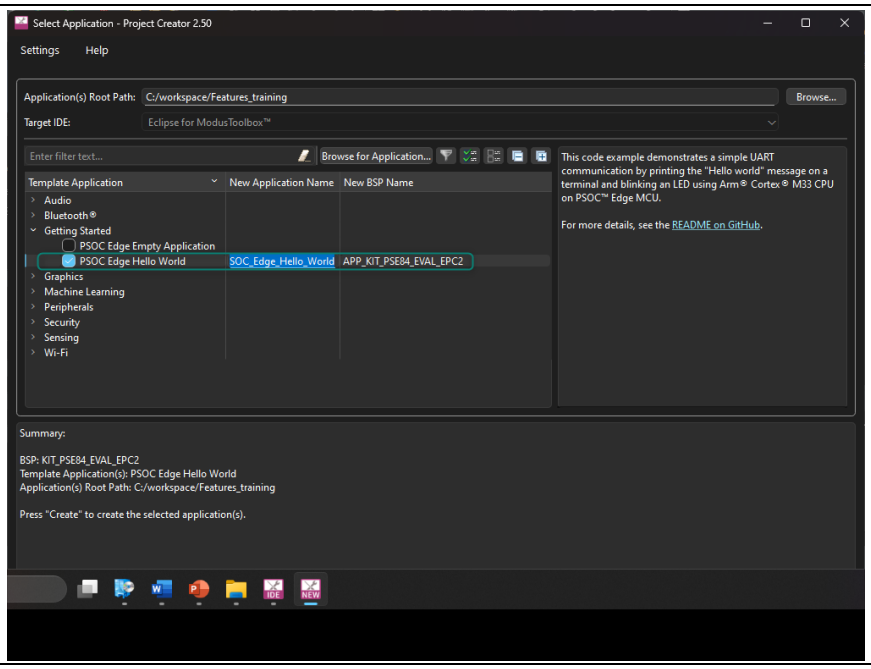


Figure 60 Selecting application

6. A new project is created, and you can see all the files in the Project Explorer window

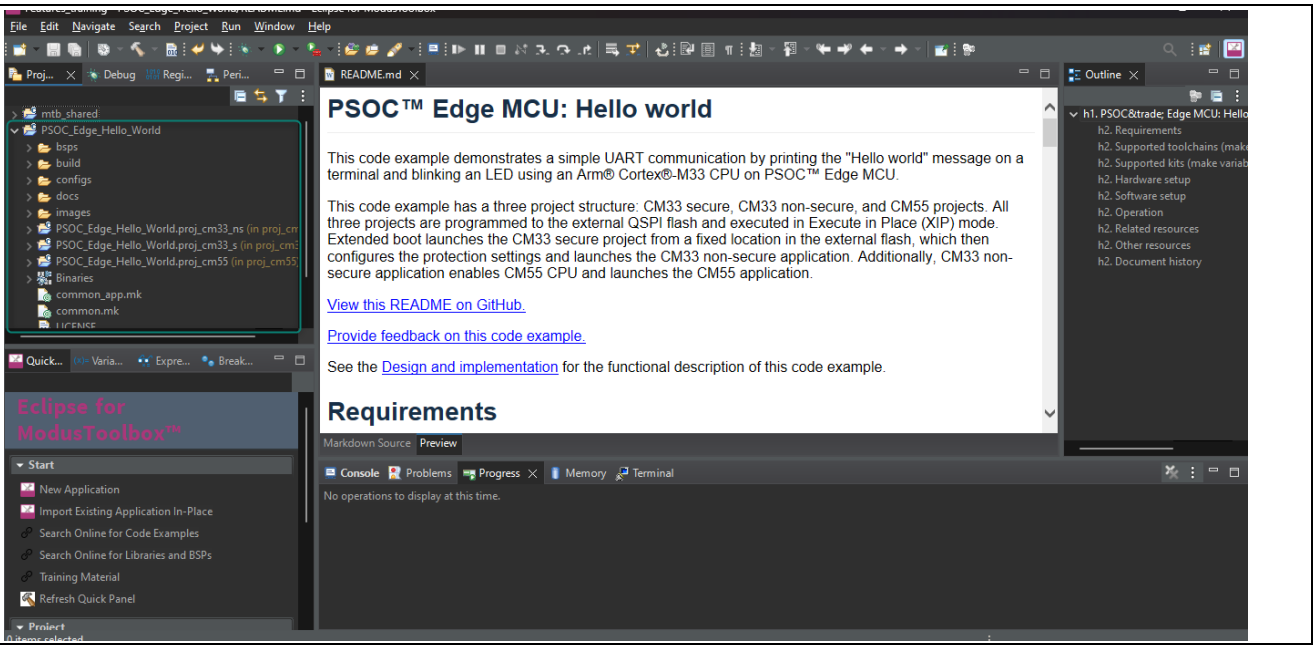


Figure 61 New application created

10 Appendix B: KIT_PSE84_EVAL details

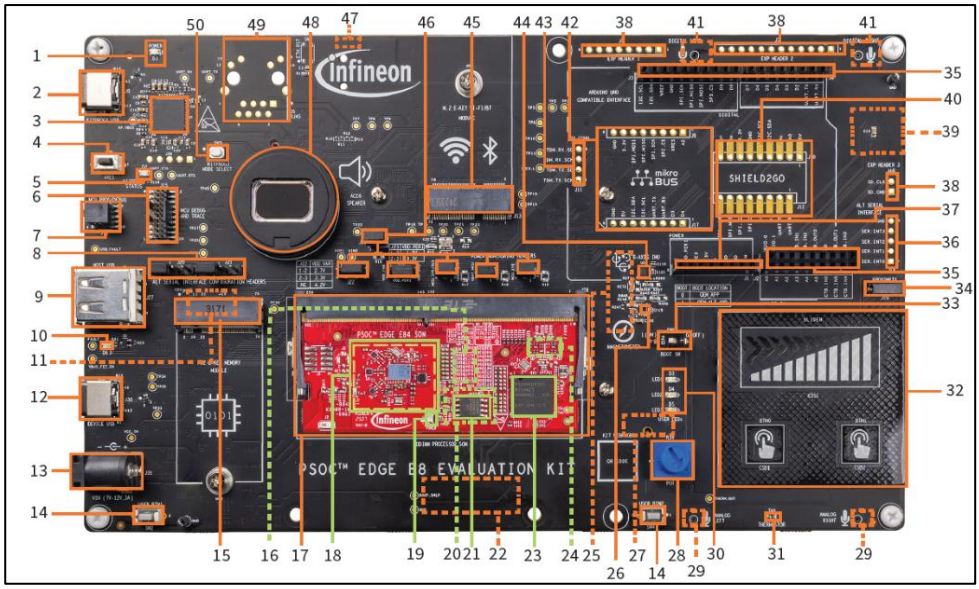


Figure 62 PSOC™ Edge EVK details

1 Base board power LED (D1)
2 KitProg3 program/debug USB Type-C connector (J8)
3 PSOC™ 5LP-based KitProg3 programmer and debugger (CY8C5868LTI-LP039, U2)
4 Reset button (SW1)
5 KitProg3 status LED (D2)
6 PSOC™ Edge E84 MCU ETM/JTAG debug and trace header (J15)
7 PSOC™ Edge E84 MCU 10-pin SWD/JTAG program and debug header (J16)
8 Alternative serial interface configuration headers (J20, J21)
9 PSOC™ Edge E84 MCU USB host Type-A connector (J27)
10 USB Type-C power delivery (PD) fault LED (D6)
11 Custom display capacitive touch panel connector (J37)**
12 PSOC™ Edge E84 MCU USB device Type-C connector (J30)
13 External power supply VIN connector (J31)
14 PSOC™ Edge E84 MCU user buttons (SW2, SW4)
15 M.2 (B-key) memory interface connector (J29)
16 128 Mbit Octal-SPI HYPERRAM™ (S70KS1283GABHI020, U12)***
17 Processor System on Module (SoM) 260-pin SODIMM connector (J28)
18 CYW55513 tri-band (Wi-Fi & Bluetooth®) combo radio (U3) section
19 Processor System on Module (SoM) power LED (D3)
20 1 Gbit Octal-SPI NOR flash (S28HS01GTGZBH1030, U10)***
21 128 Mbit Quad-SPI NOR flash (S25FS128SAGMFB100, U11)
22 MIPI-DSI custom display connector (J38)**
23 PSOC™ Edge E84 MCU (PSE846GPS2DBZC4A, U1)
24 PSOC™ 4000T CAPSENSE™ Co-processor (U9)***
25 Raspberry Pi compatible MIPI-DSI display connector (J39)**
26 3-axis magnetometer (U4)
27 Raspberry Pi compatible display capacitive touch connector (J41)**
28 Linear potentiometer (R34)
29 Analog microphones (IM73A135V01XTSA1, U36 and U37)**
30 User LEDs (D3, D4, D5)
31 Thermistor (TH1)
32 CAPSENSE™ buttons and slider (CSB1, CSB2, CSS1)
33 BOOT configuration switch (SW6)
34 Proximity sense connector (J19)
35 I/O headers compatible with Arduino UNO R3 (J2, J3, J4)
36 Alternative serial interface I/O header (J14)*
37 Power header compatible with Arduino UNO R3 (J1)
38 PSOC™ Edge E84 MCU expansion I/O headers (J6, J7, J40)*
39 MicroSD card holder (J35)**
40 Infineon's Shield2Go interface headers (J10, J12)*
41 PDM microphones (IM72D128V01XTMA1, U7 and U8)**
42 mikroBUS compatible headers by Mikroelektronika (J9, J17)*
43 Extended I2S header (J11)*
44 6-axis accelerometer and gyroscope IMU (U5)
45 M.2 (E-key) radio interface connector (J13)
46 PSOC™ Edge E84 MCU power selection/ monitoring headers (J18, J22, J23, J24, J25, J26)
47 Headphone connector (J34)*
48 Speaker (ACC6)
49 RJ45 ethernet magjack connector (J5)*
50 KitProg3 programming mode selection button (SW3)

*Footprint only, not populated on the board
**Component at the bottom side of the Baseboard
***Component at the bottom side of the SoM

Revision history

Document revision	Date	Description of changes
**	2026-03-09	Initial release

Trademarks
All referenced product or service names and trademarks are the property of their respective owners.

Edition 2026-03-09

Published by

Infineon Technologies AG

Am Campeon 1-15

85579 Neubiberg

Germany

© 2026 Infineon Technologies AG. All Rights Reserved.

Do you have a question about this document?

Email:

erratum@infineon.com

Important Notice

Products which may also include samples and may be comprised of hardware or software or both ("Product(s)") are sold or provided and delivered by Infineon Technologies AG and its affiliates ("Infineon") subject to the terms and conditions of the frame supply contract or other written agreement(s) executed by a customer and Infineon or, in the absence of the foregoing, the applicable Sales Conditions of Infineon. General terms and conditions of a customer or deviations from applicable Sales Conditions of Infineon shall only be binding for Infineon if and to the extent Infineon has given its express written consent.

For the avoidance of doubt, Infineon disclaims all warranties of non-infringement of third-party rights and implied warranties such as warranties of fitness for a specific use/purpose or merchantability.

Infineon shall not be responsible for any information with respect to samples, the application or customer's specific use of any Product or for any examples or typical values given in this document.

The data contained in this document is exclusively intended for technically qualified and skilled customer representatives. It is the responsibility of the customer to evaluate the suitability of the Product for the intended application and the customer's specific use and to verify all relevant technical data contained in this document in the intended application and the customer's specific use. The customer is responsible for properly designing, programming, and testing the functionality and safety of the intended application, as well as complying with any legal requirements related to its use.

Unless otherwise explicitly approved by Infineon, Products may not be used in any application where a failure of the Products or any consequences of the use thereof can reasonably be expected to result in personal injury. However, the foregoing shall not prevent the customer from using any Product in such fields of use that Infineon has explicitly designed and sold it for, provided that the overall responsibility for the application lies with the customer.

Infineon expressly reserves the right to use its content for commercial text and data mining (TDM) according to applicable laws, e.g. Section 44b of the German Copyright Act (UrhG).If the Product includes security features:

Because no computing device can be absolutely secure, and despite security measures implemented in the Product, Infineon does not guarantee that the Product will be free from intrusion, data theft or loss, or other breaches ("Security Breaches"), and Infineon shall have no liability arising out of any Security Breaches.

If this document includes or references software:

The software is owned by Infineon under the intellectual property laws and treaties of the United States, Germany, and other countries worldwide. All rights reserved. Therefore, you may use the software only as provided in the software license agreement accompanying the software.

If no software license agreement applies, Infineon hereby grants you a personal, non-exclusive, non-transferable license (without the right to sublicense) under its intellectual property rights in the software (a) for software provided in source code form, to modify and reproduce the software solely for use with Infineon hardware products, only internally within your organization, and (b) to distribute the software in binary code form externally to end users, solely for use on Infineon hardware products. Any other use, reproduction, modification, translation, or compilation of the software is prohibited. For further information on the Product, technology, delivery terms and conditions, and prices, please contact your nearest Infineon office or visit https://www.infineon.com